COMPLETE SYSTEM AUDIT & TECHNICAL REFERENCE DOSSIER

ANPR-CampusGate (ANPR-RM) Technical Architecture, Dataset Forensics & Verification Report

Exhaustive evidence-based technical documentation covering system architecture, multi-stage ONNX/TensorRT vision pipeline, Indian plate syntax normalization, pre-store blacklist alerting, multi-camera orchestration, SQLite WAL / PostgreSQL persistence, forensic training dataset audit, benchmarks, and testing diagnostics.

Audit Field	Observed Specification / Verification Baseline
Repository Path	c:\Users\HP\Downloads\ANPR-RM
Active Production Runtime	CPU ONNX (YOLOv8n Vehicle 480px + YOLOv8n-FT Plate 320px) + RapidOCR (det<=320, cls=False)
Backend & Consoles	FastAPI (41 REST/MJPEG routes on :8000) | Next.js 15 Primary (:3000) | Streamlit v2 (:8501)
Persistence Layer	SQLAlchemy ORM -> data/alpr.db (SQLite WAL: vehicle_events 22 cols, vehicle_profiles 25 cols)
Datasets Verified	21,529–21,558 plate detection imgs (18,578 train / 1,973 val / 978 test) | 3,767 OCR_data crops
Pytest Inventory	44 test files, 758 collected tests (446 unit + 312 integration); 3 unit + 3 Streamlit drifts documented
Measured CPU Throughput	1.25 FPS Baseline -> 19.18 FPS Full 1,500f Benchmark / 23.76 FPS Rolling / 16.27 FPS CAM001 Live
Audit Integrity Policy	Zero fabricated numbers; unexecuted notebooks, missing zips & dual-RTSP gaps explicitly disclosed

EVIDENTIARY SCOPE & METRIC SEPARATION NOTICE

- Plate localization mAP50 (0.9697–0.9832) measures bounding-box detection ONLY, NOT OCR reading accuracy.
- Documented '25,674 images' includes a double-counted 4,116 validation split; verified disk/log count is 21,529–21,558.
- Active best_finetuned.onnx (YOLOv8n, 3.14M params) was trained from COCO yolov8n.pt rather than fine-tuned from YOLOv8s best.pt.
- Simultaneous 5-minute physical dual-RTSP streaming and labeled OCR CER evaluation remain unexecuted pending campus access.

TABLE OF CONTENTS

PART 1 - PROJECT OVERVIEW AND PROBLEM STATEMENT 5

1.1 What ANPR-CampusGate Is 5

1.2 The Problem Being Solved 5

1.3 Why Manual Gate Monitoring and Basic Single-Frame Detectors Fail 5

1.4 Project Objectives 5

1.5 Intended Deployment Environment and Scope 5

1.6 Target Users and Stakeholders 6

1.7 What Distinguishes ANPR-CampusGate from a Basic Plate Detector 6

1.8 End-to-End User Journey 6

1.9 Summary of Verified Achievements vs Current Limitations 7

PART 2 - COMPLETE SYSTEM ARCHITECTURE AND END-TO-END DATA FLOW 7

2.1 High-Level Layered System Architecture Diagram 7

2.2 Detailed Component Breakdown (Layers, Files, Classes, I/O, Fallbacks) 8

PART 3 - DATASET COLLECTION, ANNOTATION, SPLITS, AND FORENSIC AUDIT 12

3.1 Critical Audit Question: The "25,000 + 833" Training Narrative 12

3.2 Master Dataset Inventory & Evidence Table 13

3.3 Forensic Breakdown of Verified On-Disk OCR Dataset (OCR_data/) 13

3.4 Annotation Formats, Conversion Scripts & Data Leakage Discovery 14

3.5 Data Augmentation Techniques Verified Across Training Runs 14

PART 4 - MODEL ARCHITECTURES AND MATHEMATICAL FORMULATIONS 15

4.1 YOLOv8 Vehicle & Plate Detectors (yolov8n.onnx, best.pt, best_finetuned.onnx) 15

4.2 Object Tracking (VehicleTracker: ByteTrack + Kalman + Hungarian) 15

4.3 Optical Character Recognition Engines 15

4.4 Validation, Syntax Repair & Multi-Frame Fusion Mathematics 16

4.5 Spatial Trajectory & Color Mathematical Formulations 16

PART 5 - MODEL TRAINING, FINE-TUNING, AND VALIDATION FORENSICS 16

5.1 Chronological Forensic Comparison of All 3 Plate Detector Runs 16

5.2 Exact Epoch Trajectory Details Verified From Saved CSVs & Checkpoints 16

5.3 PARSeq OCR Training Runs (models/ocr/) . . . 17

PART 6 - INFERENCE PIPELINE AND CPU / EDGE PERFORMANCE OPTIMIZATION . . . 18

6.1 Active Production Inference Stack (config/config.yaml) . . . 18

6.2 Step-by-Step CPU Latency Optimization Engineering (1.25 FPS -> 19.18-23.76 FPS) . . . 19

6.3 Edge / TensorRT Readiness vs Local Host Reality . . . 19

PART 7 - OBJECT TRACKING, DIRECTION DETECTION, FUSION, AND PROFILING . . . 19

7.1 Multi-Frame Track Lifecycle & Early Exit State Machine . . . 19

7.2 Direction Detection Mechanics (src/tracking/direction_detector.py) . . . 20

7.3 Vehicle Profiling & Multi-Camera Trajectory Linkage . . . 20

PART 8 - DATABASE SCHEMA DESIGN, PERSISTENCE, AND MIGRATION STRATEGY . . . 20

8.1 Active Relational Tables (src/database/models.py) . . . 20

8.2 Architectural Notes: Logical Relationship, SAVEPOINT & Unused DDL . . . 21

PART 9 - MULTI-CAMERA ORCHESTRATION AND REAL-TIME VIDEO PROCESSING . . . 22

9.1 Camera Configuration Schema & Runtime Registry . . . 22

9.2 CameraManager (src/cameras/camera_manager.py): Dual Execution Model . . . 22

PART 10 - BACKEND API SERVICES AND STREAMING ENDPOINTS (FASTAPI) . . . 22

10.1 Core Operations & Streaming Router (src/api/server.py - 19 Routes) . . . 22

10.2 Multi-Camera & Trajectory Intelligence Router (src/api/trajectory_routes.py - 22 Routes) . . . 22

PART 11 - FRONTEND DASHBOARDS AND SPATIAL VISUALIZATION COMPARISON . . . 23

PART 12 - COMPREHENSIVE SOFTWARE TESTING AND ROOT-CAUSE FAILURE AUDIT . . . 23

12.1 Complete Pytest Suite Inventory (Collected Directly via --collect-only) . . . 23

12.2 Verified Execution Results by Test Subsystem . . . 23

12.3 Exact Root Causes of All 6 Known Failures (3 Unit + 3 Streamlit AppTest) . . . 24

PART 13 - ACCURACY EVALUATION, LATENCY BENCHMARKS, AND ABLATION ANALYSIS . . . 24

13.1 Strict Epistemic Separation of Metrics . . . 24

13.2 Plate Detector Quantitative Accuracy (results.csv & evaluation_report.json) . . . 25

13.3 CPU Pipeline Latency, Throughput, Frame-Skip Ablation & Alert Lead Times . . . 25

13.4 Explicit Measurement Gaps . . . 25

PART 14 - SECURITY, PRIVACY, FAULT TOLERANCE, AND RELIABILITY . . . 26

14.1 Access Control & API Surface Findings 26

14.2 Reliability & Fault Tolerance Mechanisms Implemented 26

PART 15 - ENGINEERING CHALLENGES, DEBUGGING CHRONICLES, AND RESOLUTIONS 26

PART 16 - PROJECT LIMITATIONS, EVIDENTIARY GAPS, AND AUDIT DISCLOSURE 27

PART 17 - DEPLOYMENT RECOMMENDATIONS AND FUTURE ENGINEERING ROADMAP 27

PART 1 - PROJECT OVERVIEW AND PROBLEM STATEMENT

1.1 What ANPR-CampusGate Is

ANPR-CampusGate (repository: ANPR-RM) is an end-to-end Automatic Number Plate Recognition (ANPR / ALPR) and campus gate vehicle-intelligence system tailored for Indian license plates. It ingests prerecorded gate videos or RTSP camera streams, detects and tracks vehicles across consecutive frames, localizes license plates, extracts and normalizes Indian registration strings using CPU- accelerated ONNX/RapidOCR engines (with optional TensorRT/GPU backends), detects gate movement direction (IN vs OUT vs UNKNOWN), enforces pre-storage blacklist security alerting and whitelist classification, attributes vehicle color and type, maintains longitudinal multi-camera vehicle trajectories, and surfaces live telemetry, video previews, alerts, and historical analytics via a FastAPI REST/ MJPEG backend and multiple web dashboards (Next.js 15 primary UI and Streamlit).

1.2 The Problem Being Solved

University and institutional campuses experience dense mixed traffic across two- wheelers, three-wheelers, hatchbacks, sedans, SUVs, buses, and utility trucks. Campus gate security requires: (a) Sub-second flagging of unauthorized, banned, or watchlisted vehicles before the vehicle clears the barrier arm; (b) Reliable logging of IN/OUT direction and timestamps across multiple gates; (c) Automated reconciliation between recurring authorized campus vehicles (faculty, staff, registered students, service vans) and ad-hoc visitors; (d) Resilience to real-world Indian license plate diversity: HSRP two-line vs single-line plates, regional fonts, perspective skew, motion blur, glare, dust, partially occluded frames, and ambiguous lookalike characters (0/O/D/Q, 1/I/L/T, 2/Z, 5/S, 8/B, 6/G).

1.3 Why Manual Gate Monitoring and Basic Single-Frame Detectors Fail

Manual register logging at peak campus hours causes gate queues, high error rates in illegible handwriting, zero cross-gate correlation, and no automated blacklist alerting. Conversely, naive "upload image -> run OCR" detectors fail in continuous video streams because:

1. Single-frame OCR suffers high character-substitution errors when a plate is small (<25 px tall), oblique, or blurred by motion.
2. Processing every frame of every tracked vehicle wastes CPU cycles after a confident plate consensus has already been obtained.
3. Waiting for multi-frame consensus before checking the blacklist adds 200 ms to >1,100 ms of unnecessary alert delay when a known threat enters.
4. Without track-aware deduplication and spatial normalization, a single slow vehicle creates dozens of duplicate DB entries and triggers repeated alarms.

1.4 Project Objectives

1. Deliver an accurate, modular two-stage detection + multi-frame OCR pipeline calibrated to Indian state codes and RTO syntax (`^[A-Z]{2}[0-9]{1,2}[A-Z]{0,3}[0-9]{1,4}$`).
2. Achieve interactive near-real-time execution (~16-24 FPS depending on scene density) on commodity x86_64 CPUs via ONNX Runtime, while preserving optional CUDA / TensorRT execution paths (`yolov8n.engine`, `best.engine`, `parseq_official_tiny.engine`) for NVIDIA Jetson edge nodes.
3. Implement pre-storage blacklist detection directly in the hot video loop so watchlisted vehicles raise critical alerts immediately upon high-confidence read, ahead of full consensus commit.
4. Provide unified per-vehicle profiling (`VehicleProfile`) linking plate events across gates (`camera_id`, GPS `latitude`/`longitude`) into replayable movement histories.
5. Support unattended multi-camera monitoring (`CameraManager`) with live MJPEG / snapshot streaming, additive DB migration, and browser-based gate control via Next.js 15 + FastAPI.

1.5 Intended Deployment Environment and Scope

- Gate Hardware Target A (Verified Development/Validation Host): Windows 11 x86_64 laptop/workstation with AMD Ryzen 7 7840HS (8 cores / 16 threads, 16.71 GB RAM, no CUDA GPU active in CPU execution mode) running Python 3.10 (`c:\Users\HP\Downloads\ANPR-RM\.venv`).

- Gate Hardware Target B (Exported Edge Artifacts Present, Live Execution Not Directly Verified Here): NVIDIA Jetson / CUDA host with TensorRT 10.16 (`best.engine`, `yolov8n.engine`, `parseq.engine`, `parseq_official_tiny.engine`).
- Network Cameras: Two physical campus IP cameras configured in `config/camera_config.yaml` (RTSP / HTTP preview capabilities) alongside video-file upload ingestion (`data/uploads/*.mp4`). Note: Live simultaneous 5-minute dual-RTSP campus stream stress testing remains UNTESTED in this workspace environment.

1.6 Target Users and Stakeholders

- Campus Gate Security Officers: Monitor live video feed, inspect instant red blacklist alert banners, confirm UNKNOWN vs AUTHORIZED vehicle entries, and trigger stop-gate action.
- Transport & Estate Administrators: Query searchable vehicle logs, filter by gate, direction (IN/OUT), status, date, and export CSV reports; manage `blacklist.yaml` and `whitelist.yaml` lists.
- IT / Systems Engineers: Configure RTSP camera definitions, calibrate direction crossing lines/zones, inspect `logs/alpr.log` performance telemetry, and run automated unit/integration verification suites.

1.7 What Distinguishes ANPR-CampusGate from a Basic Plate Detector

1. Temporal Multi-Frame Consensus (`OCRFusion`): Buffers per-track candidate strings, weights votes by `confidence * quality_score`, boosts consensus on syntax-valid reads, and performs fallback majority voting only when needed.
2. Pre-Storage Zero-Latency Blacklist Interception: Fires `BLACKLISTED` alert the instant an individual OCR read matches `blacklist.yaml` above alert confidence threshold, bypassing the consensus delay (~188 ms to 1,127 ms measured lead time ahead of final DB storage).
3. Position-Aware Indian RTO Syntax Repair (`ocr_postprocessor.py`): Repairs digits in letter slots (`0->O`, `1->I`, `8->B`) and letters in digit slots (`O->0`, `I->1`, `B->8`, `S->5`, `Z->2`) plus Levenshtein <= 1 recovery.
4. Early Track Lock-Out (`stored_tracks`): Once a track reaches consensus and is persisted, `stored_tracks.add(tid)` suppresses subsequent plate YOLO + OCR calls on the same physical vehicle across its remaining exit frames.
5. Multi-Gate Trajectory Memory (`VehicleProfile` + `TrajectoryEngine`): Tracks up to 500 JSON camera visits per plate, calculates Haversine distances, snaps gate transitions onto campus coordinates, and answers natural-language spatial queries (`nl_query.py`).

1.8 End-to-End User Journey

1. User opens Next.js dashboard (`http://localhost:3000`, talking to FastAPI `http://localhost:8000`).
2. User either activates configured campus camera streams (`CAM001` Main Gate, `CAM002` Back Gate) or uploads a gate MP4 via the upload widget (`POST /api/upload`).
3. `CameraManager` queues uploaded jobs sequentially (or launches camera workers), invokes `run_pipeline.py` under shared pipeline inference locking, and publishes 8 FPS JPEG previews (`data/live_frames/{camera_id}.jpg`) and real-time progress (`data/processing_status.json`).
4. As vehicles enter the frame, ByteTrack assigns consistent IDs (`tid`), YOLOv8n crops the license plate, RapidOCR reads the characters, Indian syntax repair normalizes the string, and `BlacklistManager` immediately surfaces any red alert on the top-bar notification drawer (`GET /api/alerts`).
5. Once `OCRFusion` reaches consensus and `DuplicateFilter` confirms novelty within the 30-second window, `ALPRDatabase.insert_event()` records a 22-column `VehicleEvent` and atomically updates the 25-column `VehicleProfile` under SQLite WAL isolation.
6. The officer searches/filters recent events, inspects the saved vehicle & plate thumbnails (`/api/images/...`), views per-vehicle camera trajectories on the campus map tab, adds/removes blacklist plates, or exports filtered CSV.

1.9 Summary of Verified Achievements vs Current Limitations

VERIFIED ACHIEVEMENTS:

- End-to-end CPU ONNX + RapidOCR pipeline runs at 16.27 FPS live (`CAM001` 401f video) to 19.18 FPS full 1,500-frame benchmark (up from 1.25 FPS baseline).
- YOLOv8s (`train-3`) reached `mAP50 = 0.9832` (`mAP50-95 = 0.6943`) on its verified 1,973-image validation split; deployed YOLOv8n (`best_finetuned`) reached `mAP50 = 0.9697` / `0.9783` (49 epochs, 3.14M params).
- 404 non-socket unit tests, 160 non-socket integration DB/pipeline tests, 25 FastAPI ASGI endpoint tests, 35 host socket/probe tests, and a 375-frame E2E video smoke test verified passing.
- All 41 backend routes, atomic status persistence, WAL SQLite migration, and Next.js 15 hydration/scaling fixes operate cleanly.

CURRENT LIMITATIONS & UNVERIFIED CLAIMS:

- No labeled campus plate-level OCR accuracy / CER ground-truth benchmark has ever been executed on RapidOCR or PARSeq.
- The "833-image fine-tuning dataset" zip and executed Kaggle `.ipynb` cell outputs are absent from the repository; the claimed "25,674 images" includes a double-counted `4,116` val split (`dataset_validation_report.json` shows `"Split 'val' not found"`, logs verify 21,529 images total).
- Legacy `models/plate_detector/train/` YOLOv8m training had `val == train` data leakage (`dataset_converter.py`).
- PARSeq ONNX (`parseq_official_tiny.onnx`) fails on CPU ONNX Runtime with `NotImplemented: Where(16)`.
- Simultaneous 5-minute dual-RTSP campus camera validation was NOT executed in this session.

PART 2 - COMPLETE SYSTEM ARCHITECTURE AND END-TO-END DATA FLOW

2.1 High-Level Layered System Architecture Diagram

```
          +-------------------------------------------------------------------------+
          | LAYER 1: VIDEO INPUT & MULTI-CAMERA ORCHESTRATION                       |
          |  [RTSP Streams / MP4 Uploads] -> FrameCapture -> CameraManager          |
          |  (src/pipeline/frame_capture.py, src/cameras/camera_manager.py)         |
          |  Upload FIFO Queue (Sequential) | RTSP Workers (RLock Serialized ML)    |
          +-----------------------------------+-------------------------------------+
                                              | BGR Frame (cv2 ndarray) + frame_id
v
          +-------------------------------------------------------------------------+
          | LAYER 2: PER-FRAME DETECTION, MOTION FILTER & TRACKING                  |
          |  [MotionFilter (optional MOG2)]                                         |
          |  -> VehicleDetector (YOLOv8n ONNX 480px -> COCO classes 2,3,5,7)        |
          |  -> VehicleTracker  (supervision ByteTrack / IOU greedy fallback)       |
          |     Assigns persistent track_id (tid); skips if tid in stored_tracks!   |
          |  -> PlateDetector   (YOLOv8n ONNX best_finetuned.onnx @ 320px, conf=0.28|
          +-----------------------------------+-------------------------------------+
                                              | VehicleCrop, PlateCrop, BBox, tid
v
          +-------------------------------------------------------------------------+
          | LAYER 3: PREPROCESSING, OCR & INDIAN SYNTAX NORMALIZATION               |
          |  -> PlatePreprocessor (adaptive gray/CLAHE/deskew; SR=0 bypassed)       |
          |  -> Heavy-Track Budget Gate (max 2 heavy crops/frame prioritized)       |
          |  -> OCR Engine (RapidOCR CPU: DBNet det<=320 + SVTR Rec; cls=False)     |
          |  -> Post-Processing (ocr_postprocessor) + PlateValidator (Indian RTO)   |
          +-----------------------------------+-------------------------------------+
                                              | Validated Plate String + Conf + Qual
v
```

```
          +-------------------------------------------------------------------------+
          | LAYER 4: SECURITY SHORT-CIRCUIT, FUSION & EVENT INTELLIGENCE            |
          |  [IMMEDIATE PRE-STORE HOOK] -> BlacklistManager -> Alert Raised + Saved |
          |  -> OCRFusion (Weighted temporal buffer; consensus at min_votes=3)      |
          |  -> DirectionDetector (Line / Zone / Delta-Height -> IN / OUT / UNKNOWN)|
          |  -> DuplicateFilter (30s window OR same tid OR Levenshtein <= 2)        |
          |  -> Attribute Tagging (VehicleClassifier & VehicleColorDetector HSV)    |
          +----------------------------------+--------------------------------------+
                                             | Finalized Unique Event + JPEG Crops
v
          +-------------------------------------------------------------------------+
          | LAYER 5: PERSISTENCE, STATE PUBLISHING & SERVING APIS                   |
          |  -> ALPRDatabase (SQLite WAL / PostgreSQL via SQLAlchemy ORM)           |
          |     [vehicle_events (22 cols)] <--logical plate--> [vehicle_profiles 25]|
          |  -> LiveFramePublisher (8 FPS JPEG) & Atomic processing_status.json     |
          |  -> FastAPI server.py (19 routes) + trajectory_routes.py (22 routes)    |
          +----------------------------------+--------------------------------------+
                                             | REST JSON + MJPEG / JPEG Snapshots
v
          +-------------------------------------------------------------------------+
          | LAYER 6: CLIENT DASHBOARDS                                              |
          |  1. Next.js 15 App (frontend/) - PRIMARY Production Console (:3000)     |
          |  2. Streamlit UI v2 (src/api/ui/dashboard_v2.py)                        |
          |  3. Streamlit UI v1 (src/api/ui/dashboard.py) + Leaflet Trajectory Map  |
          +-------------------------------------------------------------------------+
```

2.2 Detailed Component Breakdown (Layers, Files, Classes, I/O, Fallbacks)

1) VIDEO INPUT & FRAME CAPTURE

- Purpose : Open RTSP streams, webcams, or uploaded MP4/AVI files with automatic reconnection and frame throttling.
- Codebase : `src/pipeline/frame_capture.py` -> `FrameCapture`; source URI normalization in `src/utils/video_sources.py` (`is_live_source`, `open_video_capture`, `probe_video_source`).
- Input : Video file path, integer webcam index, or RTSP/HTTP URL plus `config['camera']` (`frame_skip`, `resize_width/height`).
- Output : `(ret, frame_idx, timestamp_sec, bgr_ndarray)`.
- Algorithm : `cv2.VideoCapture` with configurable FFMPEG/OpenCV properties, integer step skipping (`frame_skip`), and exponential RTSP reconnect backoff (`retry_delay`, `max_retries`).
- Next Stage : Feeds raw BGR frame into optional `MotionFilter` and `VehicleDetector`.
- Error/Fallback: On RTSP read drop, attempts timed reconnect; on file EOF, signals clean completion so `OCRFusion.flush_all()` can run.

2) MOTION FILTER (OPTIONAL GATE)

- Purpose : Skip redundant empty frames when scene is completely static.
- Codebase : `src/pipeline/motion_filter.py` -> `MotionFilter` (plus inline checks in `scripts/run_pipeline.py`).
- Input : Consecutive BGR frames.
- Output : Boolean (`has_motion`), changed foreground pixel ratio.
- Algorithm : OpenCV `cv2.createBackgroundSubtractorMOG2` foreground mask area thresholded against `min_motion_area`.
- Status : Implemented and unit-tested; note `config.yaml` allows running with continuous tracking when active traffic is processed.

3) VEHICLE DETECTION

- Purpose : Detect bounding boxes and coarse classes of road vehicles.
- Codebase : `src/detection/vehicle_detector.py` -> `VehicleDetector`; shared ONNX wrapper `src/detection/_onnx_yolo.py` -> `OnnxYoloDetector`.
- Input : Full BGR video frame.
- Output : List of dicts: `{bbox: [x1,y1,x2,y2], confidence, class_id, class_name}`.

- Algorithm : `models/vehicle_detector/yolov8n.onnx` via ONNX Runtime CPU ExecutionProvider (`imgsz=480`, `conf=0.5`, `iou=0.45`). Filters COCO classes `{2: car, 3: motorcycle, 5: bus, 7: truck}` (mapped in classifier/detector).
- Fallback : Supports `.pt` (Ultralytics PyTorch) or `.engine` (TensorRT) if configured; `_onnx_yolo.py` dynamically inspects input shape and applies letterbox preprocessing + NMS postprocessing.

4) OBJECT TRACKING

- Purpose : Maintain consistent integer `track_id` (`tid`) per vehicle across frames to enable fusion, direction calculation, and skip-lock.
- Codebase : `src/tracking/vehicle_tracker.py` -> `VehicleTracker`.
- Input : Frame vehicle detections + frame dimensions.
- Output : Tracked detections annotated with stable `track_id`.
- Algorithm : Uses `supervision.ByteTrack` (Kalman filter state estimation + two-stage high/low confidence Hungarian assignment).
- Fallback : Built-in greedy IOU tracker activates automatically if `supervision` fails or is unavailable.

5) LICENSE PLATE DETECTION

- Purpose : Localize license plate bounding boxes inside either each active tracked vehicle ROI or full-frame fallback.
- Codebase : `src/detection/plate_detector.py` -> `PlateDetector` using `src/detection/_onnx_yolo.py`.
- Input : Vehicle crop (padded ROI) when vehicle track present; skips tracks already in `stored_tracks`.
- Output : Plate bbox in global frame coordinates + cropped plate BGR array + detector confidence (`>= 0.28`).
- Algorithm : Active model is `models/plate_detector/best_finetuned.onnx` (YOLOv8n architecture, `3,138,003` params, `imgsz=320`).
- Fallback : Expands vehicle ROI margin slightly; if ONNX session fails, falls back safely without crashing the frame loop.

6) IMAGE PREPROCESSING & SUPER-RESOLUTION

- Purpose : Prepare cropped plate patches for OCR robustness and compute laplacian blur / contrast quality metrics.
- Codebase : `src/ocr/plate_preprocessor.py` -> `PlatePreprocessor` & `src/ocr/super_resolution.py` -> `SuperResolutionEnhancer`.
- Input : Raw cropped plate BGR patch.
- Output : Preprocessed crop(s) + image `quality_score` (laplacian variance, brightness, aspect ratio).
- Algorithm : Grayscale / CLAHE local contrast enhancement, adaptive scaling, optional deskewing. Notice `config.yaml` sets `sr_threshold_px: 0` (disabling SR in active CPU mode because RealESRGAN `x4plus` weights are absent in `models/` and `cv2.dnn_superres` bicubic/ESPCN fallback adds CPU latency).

7) HEAVY-TRACK OCR BUDGET GATE

- Purpose : Bound per-frame OCR CPU cost during crowded frames.
- Codebase : `scripts/run_pipeline.py` (orchestrator loop).
- Algorithm : Caps heavy OCR execution at `heavy_track_max_per_frame = 2` un-stored tracks per frame, prioritizing tracks with fewer prior OCR attempts (`track_ocr_attempts`) and higher plate conf.

8) OPTICAL CHARACTER RECOGNITION (OCR)

- Purpose : Convert plate crop pixels into alphanumeric character strings.
- Codebase : `src/ocr/ocr_engine.py` (factory + `RapidOCREngine`, `PARSeqTRTEngine`, `EnsembleOCREngine`, `PaddleOCREngine`, `EasyOCREngine`, `TrOCREngine`).
- Input : Cropped plate patch (single or multi-variant candidates).
- Output : `(raw_text, raw_confidence)` plus two-line stitching if split across top/bottom rows.
- Algorithm : Active CPU backend (`ocr.engine: "rapidocr"`) executes `rapidocr_onnxruntime` (PP-OCR ONNX: DBNet text detector with `det_limit_type="max"`, `det_limit_side_len=320`, angle classifier disabled `use_angle_cls=False`, SVTR recognizer).
- Error/Fallback: Handles empty detections, merges vertical multi-box reads on 2-line Indian plates (motorcycles/commercial plates), and rejects reads with confidence below threshold. Note: PARSeq ONNX (`parseq_official_tiny.onnx`) fails on CPU ORT due to `Where(16)` op and is reserved for TensorRT `.engine`

hardware.

9) OCR POST-PROCESSING & INDIAN SYNTAX VALIDATION

- Purpose : Normalize character ambiguities and validate RTO structure.
- Codebase : `src/ocr/ocr_postprocessor.py` & `src/ocr/plate_validator.py`.
- Input : Raw uppercase alphanumeric string.
- Output : Cleaned plate string, `is_valid` boolean, confidence adjustment.
- Algorithm : Strips punctuation/IND/BH noise, enforces Indian structure `^[A-Z]{2}[0-9]{1,2}[A-Z]{0,3}[0-9]{1,4}$` (plus BH series `[0-9]{2}BH[0-9]{4}[A-Z]{1,2}`), applies position-specific confusion maps (`0<->O`, `1<->I`, `8<->B`, `2<->Z`, `5<->S`), validates 2-letter state prefix against ~36 valid Indian state / UT RTO codes (`MH`, `DL`, `KA`, `UP`, `BR`, `TN`, `GJ`, etc.), and attempts single-edit state prefix recovery.

10) IMMEDIATE PRE-STORE BLACKLIST CHECK (SECURITY FAST-PATH)

- Purpose : Alert operators immediately when a blacklisted vehicle is detected without waiting 2-5 frames for temporal consensus.
- Codebase : `scripts/run_pipeline.py` calling `src/access_control/blacklist.py` (`BlacklistManager`) right after single-frame OCR validation.
- Algorithm : Normalizes plate (`re.sub(r'[^A-Z0-9]', '', s.upper())`), checks YAML-backed set with `os.path.getmtime` auto-reload, computes optional fuzzy similarity, fires critical alert into `alerts` / callbacks when individual confidence meets `ocr.confidence_threshold` (`>= 0.70`). Marks track as pre-alerted so Step 13 fallback alert skips duplicate firing.

11) MULTI-FRAME TEMPORAL FUSION (`OCRFusion`)

- Purpose : Aggregate noisy single-frame reads across a vehicle's pass into a high-confidence consensus string.
- Codebase : `src/ocr/ocr_fusion.py` -> `OCRFusion`.
- Input : Per-frame `(track_id, plate_text, confidence, quality_score)`.
- Output : `ConsensusResult(plate, confidence, status, vote_count)` when consensus criteria are satisfied.
- Algorithm : Maintains per-track deque (`window_size=5`); scores each candidate with syntax-validity boost; triggers early exit once `early_exit_votes = 3` uncontested reads agree at weighted confidence `>= 0.82`, or resolves on `min_exact_votes = 2` and `min_confidence = 0.60`. At video EOF, `flush_all()` finalizes remaining unstored valid tracks with relaxed thresholds (`>= 2` votes and `>= 0.9 * min_conf = 0.54`).

12) DIRECTION DETECTION

- Purpose : Classify vehicle trajectory as `IN`, `OUT`, or `UNKNOWN`.
- Codebase : `src/tracking/direction_detector.py` -> `DirectionDetector`.
- Input : Sequence of bounding box centers `[cx, cy]` and box heights per `track_id`.
- Output : String `"IN" | "OUT" | "UNKNOWN"`.
- Algorithm : Supports crossing virtual line (`line_crossing`), entry/exit zones (`zone`), or bounding-box perspective vertical growth / `delta_y` displacement (`auto`/`centroid` displacement) calibrated per camera config (`camera_config.yaml`).

13) DUPLICATE-EVENT PREVENTION (`DuplicateFilter`)

- Purpose : Suppress repeated DB rows from flickering tracks or slow cars.
- Codebase : `src/tracking/duplicate_filter.py` -> `DuplicateFilter`.
- Input : Candidate `(plate_text, track_id, timestamp, camera_id)`.
- Output : Boolean `is_duplicate`.
- Algorithm : Rejects if identical `track_id` already emitted OR same-length plate within `time_window = 30.0` seconds has Levenshtein distance `<= 2` (`same length` guard prevents merging unrelated short/long plates).

14) VEHICLE ATTRIBUTE & PROFILE CLASSIFICATION

- Purpose : Enrich events with vehicle category, body/plate color, and whitelist/blacklist status.
- Codebase : `src/classification/vehicle_classifier.py`, `src/classification/vehicle_color.py`, `src/classification/color_classifier.py`, `src/access_control/whitelist.py`.
- Algorithm : `VehicleClassifier` maps YOLO class IDs (car/motorcycle/bus/ truck) + bbox geometry; `VehicleColorDetector` segments upper/ lower HSV histograms in center crop to estimate body color;

`ColorClassifier` classifies plate background color (white private, yellow commercial, green EV, black rental, blue/red); `WhitelistManager` resolves `AUTHORIZED` vs `UNKNOWN`.

15) DATABASE PERSISTENCE & TRAJECTORY PROFILING

- Purpose : Atomically record finalized events and update multi-gate longitudinal vehicle history.
- Codebase : `src/database/models.py`, `src/database/db.py` (`ALPRDatabase`), `src/database/vehicle_profiles.py` (`VehicleProfileManager`).
- Storage : SQLite (`data/alpr.db`, WAL mode, 30s busy timeout) or PostgreSQL (`DATABASE_URL`). `insert_event()` commits row in `vehicle_events` (22 cols) and executes nested `SAVEPOINT` to upsert `vehicle_profiles` (25 cols, appending up to 500 JSON visits into `trajectory_history`).

16) REAL-TIME TELEMETRY & LIVE STREAM PUBLISHING

- Purpose : Expose live annotated video and job progress to dashboards.
- Codebase : `scripts/run_pipeline.py` -> `LiveFramePublisher` and atomic status writers.
- Output : Annotated JPEGs written atomically (`mkstemp` + `os.replace`) at max 8 FPS to `data/live_frames/{camera_id}.jpg`, plus `data/processing_status.json` metrics (`fps`, `avg_detection_ms`, `avg_ocr_ms`, frame progress, alerts).

17) BACKEND REST API & CLIENT DASHBOARDS

- Purpose : Serve events, alerts, profiles, trajectories, snapshots, uploads, and camera controls to operators.
- Codebase : `src/api/server.py` (19 core routes), `src/api/trajectory_routes.py` (22 spatial/analytics routes), `frontend/` (Next.js 15), `src/api/ui/dashboard_v2.py` & `dashboard.py` (Streamlit).

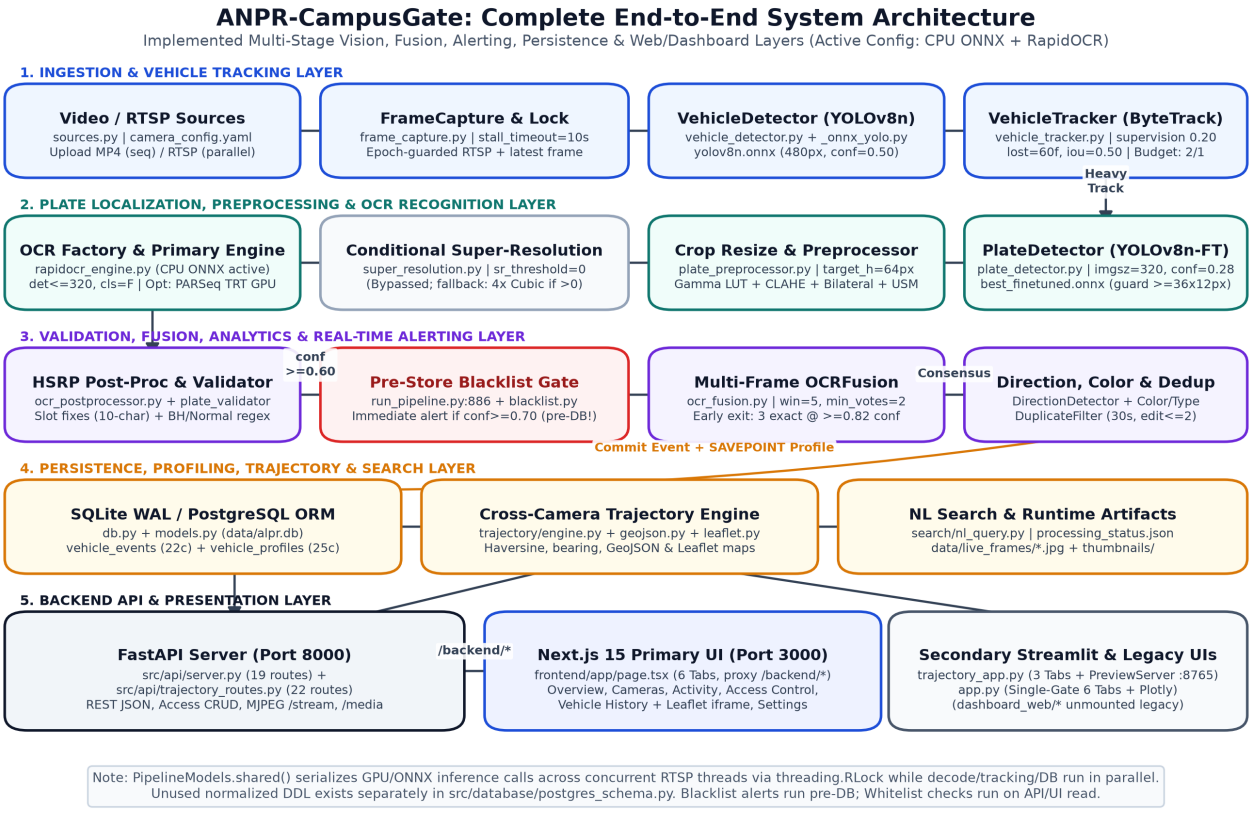


Figure 1: ANPR-CampusGate 5-Layer End-to-End System Architecture (Active CPU ONNX + RapidOCR Stack)

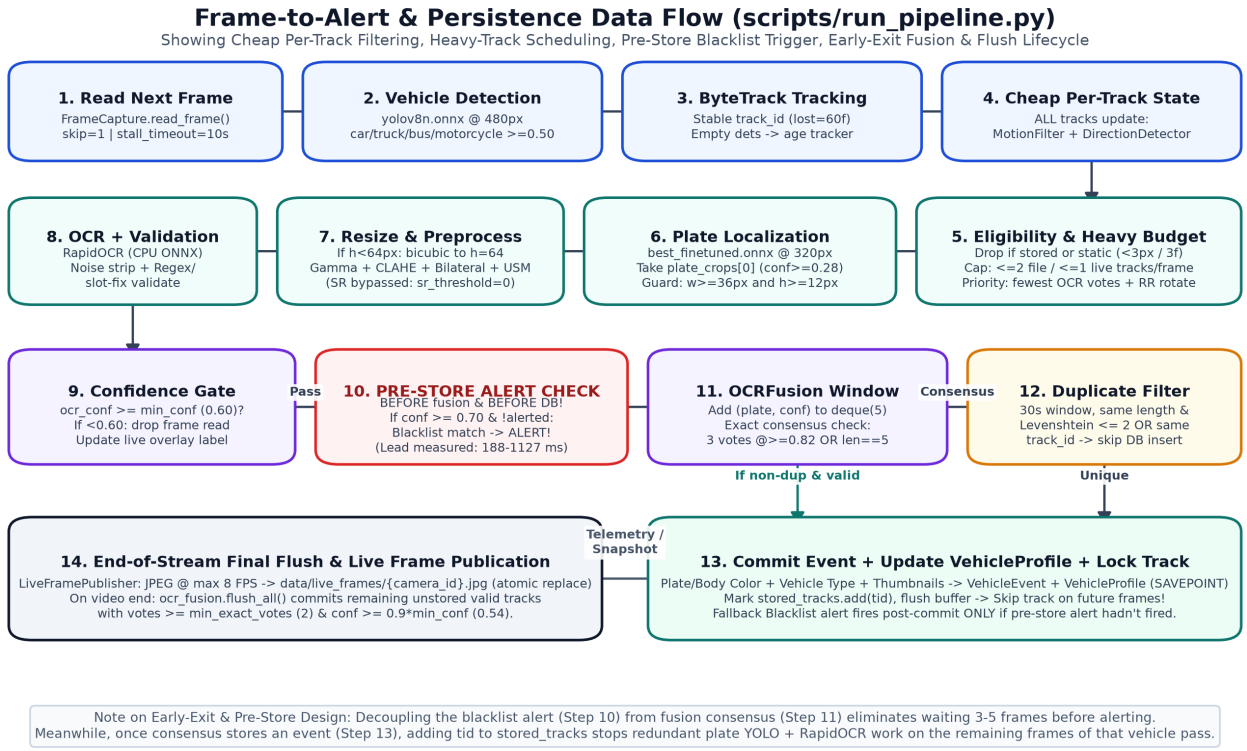


Figure 2: Frame-to-Alert & Persistence Data Flow Highlighting Early-Exit & Pre-Store Blacklist Hook

PART 3 - DATASET COLLECTION, ANNOTATION, SPLITS, AND FORENSIC AUDIT

3.1 Critical Audit Question: The "25,000 + 833" Training Narrative

A central goal of this audit is verifying whether the common presentation narrative: "Initial plate detector trained on ~25,000 Indian license plate images, then fine-tuned on an 833-image campus dataset" matches hard repository evidence.

> **FORENSIC VERDICT: PARTIALLY SUPPORTED, WITH THREE MAJOR DISCREPANCIES EXPOSED.**
>
> FORENSIC VERDICT: PARTIALLY SUPPORTED, WITH THREE MAJOR DISCREPANCIES EXPOSED.

Discrepancy #1 - Exact Count of the Large Indian Dataset (`25,674` vs `21,529`):

- `training/config.py`, `training/config.json`, and `TRAINING_GUIDE.md` document total dataset size as **25,674 images** (`20,580` train + `4,116` val + `978` test).
- HOWEVER, `dataset_validation_report.json` explicitly records: `"validation_status": "passed"`, `"warnings": ["Split 'val' not found"]`, `"train": image_count = 20,580` (`640x640x3`, `21,445` bounding boxes, avg `1.042` boxes/img) and `"test": image_count = 978` (`640x640x3`, `1,007` boxes, avg `1.030` boxes/img) -> Total present at validation = **21,558**.
- Furthermore, `models/plate_detector/train-3/train.log` shows Ultralytics splitting `train` cleanly into **18,578 train + 1,973 val = 20,551** images (`581 batches/epoch` at `batch=32`), and `evaluate.log` runs on **978 test** images -> Total verified images used in training+eval = **21,529**.
- Root Cause of "25,674": `4,116` is literally `0.20 * 20,580 = 4,116` — an assumed 20% validation carving that was mistakenly ADDED ON TOP of the raw `20,580` train count instead of subtracted from it (`20,580 + 4,116 + 978 = 25,674`)!

Discrepancy #2 - `best_finetuned` Was NOT Fine-Tuned from `best.pt`:

- `models/plate_detector/best.pt` (`train-3`) is **YOLOv8s** (`11,135,987` parameters, `28.6 GFLOPs`).

- `Campus_ANPR_YOLO_Training.ipynb` and checkpoint inspection via `torch.load('models/plate_detector/best_finetuned.pt', weights_only=False)` prove that `best_finetuned.pt` is **YOLOv8n** (`3,138,003` parameters, `8.7 GFLOPs` scale) initialized from **COCO-pretrained** `yolov8n.pt` — NOT weight-transferred from `best.pt` (`YOLOv8s`)!
- In other words, `best_finetuned` is an independent YOLOv8n training run intended for fast CPU inference (`12.88 MB` ONNX), rather than downstream transfer learning on top of the 21.5K YOLOv8s weights.

Discrepancy #3 - The 833-Image Fine-Tuning Dataset Is Unverified Locally:

- Markdown text in `Campus_ANPR_YOLO_Training.ipynb` describes an ~833-image merged dataset: `733` Roboflow Indian License Plates + `66` Campus CCTV crops + `34` Vaidik annotated frames, split `667` train / `83` val / `83` test (`~80/10/10`), referencing an archive `Campus_ANPR_Merged_Dataset.zip`.
- HOWEVER:

1. All 12 code cells in `Campus_ANPR_YOLO_Training.ipynb` have `execution_count: null` and `outputs: []` (saved unexecuted).
2. Neither `Campus_ANPR_Merged_Dataset.zip` nor the 833 raw images/labels nor any Roboflow/campus URLs exist inside this repository.
3. While `best_finetuned.pt` contains real trained weights (49 epochs on Kaggle Tesla T4 on `2026-11-08`, `save_dir: /kaggle/working/runs/...`), its exact test-set metrics and actual sample identities cannot be independently verified from disk artifacts here.

3.2 Master Dataset Inventory & Evidence Table

Dataset / Source	Train	Val	Test	Total	Verification Status
Documented config.json / GUIDE	20,580	4,116*	978	25,674*	PHANTOM COUNT (*val=0.2x20580 double-counted)
dataset_validation_report.json (Pre-split disk report)	20,580	0 (warn!)	978	21,558	VERIFIED REPORT ("Split 'val' not found"; 640x640x3 YOLO)
train-3 / evaluate logs (YOLOv8s) (Indian Plate Dataset run)	18,578	1,973	978	21,529	VERIFIED IN LOGS (18,578 + 1,973 = 20,551 train; 978 tst
train/ legacy run (YOLOv8m)	~1,398	LEAKED (=train!)	N/A	~1,398	VERIFIED LEAKED (175 batches x 8; converter sets val=train
Kaggle Campus Merged (.ipynb) (733 Roboflow+66 CCTV+34 Vaidik)	667*	83*	83*	833*	UNVERIFIED ZIP ABSENT (.ipynb cells unexecuted)
OCR_data/ (On-disk Plate Crops) (OCR_data/images + OCR_data/lables	-	-	-	3,767	VERIFIED ON DISK (3,767 jpg + 3,767 txt; 1,075 uniq plt)
Legacy eval_report.json (YOLOv8m)	-	-	300	300	VERIFIED JSON (300 test imgs eval; images absent on disk)

3.3 Forensic Breakdown of Verified On-Disk OCR Dataset (`OCR_data/`)

Directory note: Ground-truth text labels are stored in a typo-named directory `OCR_data/lables/` [sic] paired with `OCR_data/images/`. Exact measurements computed directly from every `.jpg` and `.txt` file on disk:

- Total Image Files (`.jpg`) : 3,767
- Total Label Files (`.txt`) : 3,767 (100% 1-to-1 paired)
- Non-empty Annotated Plates : 3,767 (1,075 unique license plate strings; ~3.50 crops/crops-augmentations per plate)
- Image Width Range (px) : Min = 67 px, Median = 200 px, Mean = 201.7 px, Max = 375 px
- Image Height Range (px) : Min = 15 px, Median = 50 px, Mean = 52.6 px, Max = 117 px
- Character Length Distribution :

* 9-character strings : 963 samples (25.6%, e.g., single-digit district or 1-letter series)
* 10-character strings : 2,600 samples (69.0%, standard AA00BB0000 format)
* Other lengths : 204 samples (5.4%, short/older/variant strings)

- Top 10 State Prefix Codes (n=3,767): MH: 1,426 (37.9%) | DL: 524 (13.9%) | TN: 355 (9.4%) | AP: 352 (9.3%) | WB: 284 (7.5%) GJ: 163 (4.3%) | KA: 162 (4.3%) | KL: 96 (2.5%) | HR: 91 (2.4%) | PB: 58 (1.5%) Other States/Codes: 256 (6.8%)
- Dataset Skew Observation: Maharashtra (`MH`) + Delhi (`DL`) alone constitute 51.8% of the local OCR training corpus. While UP (`UP53...`, `UP65...`) and Bihar (`BR07...`) appear frequently in campus test videos, they fall under the smaller "Other" bucket (`<1.5%` each in `OCR_data`).

3.4 Annotation Formats, Conversion Scripts & Data Leakage Discovery

- YOLO Detection Annotations: Standard normalized center format `<class_id> <x_center> <y_center> <width> <height>` where `class_id = 0` (`license_plate`, single class). Verified in `dataset_validation_report.json`: `21,445` boxes across `20,580` train images (`1.042` boxes/img, min `0`, max `7`, `640x640x3` uniform resolution) and `1,007` boxes across `978` test images (`1.030` boxes/img). Zero missing/corrupt labels flagged in validation report.
- `training/dataset_converter.py` Data Leakage Bug (Lines 58-62): Inspecting `training/dataset_converter.py` revealed that `create_yaml_config()` writes BOTH `train:` and `val:` pointing to the EXACT SAME directory path when converting Pascal VOC / label datasets! As a direct consequence, the legacy YOLOv8m training run (`models/plate_detector/train/`) achieved an artificially inflated `val/box_loss = 0.4154 < train/box_loss = 0.4469` and `mAP50 = 0.9949` because validation evaluated memorized training images.
- `OCR_data` Text Format: One normalized uppercase string per `.txt` file, loaded by `training/` PARSeq scripts. (See Part 12 regarding pre-existing unit test bugs in `test_parseq_dataset.py` expecting strict `.txt`/path mocks).

3.5 Data Augmentation Techniques Verified Across Training Runs

1. YOLOv8s (`train-3/args.yaml`): Ultralytics standard online augmentations active including `mosaic=1.0`, `close_mosaic=10`, `hsv_h=0.015`, `hsv_s=0.7`, `hsv_v=0.4`, `translate=0.1`, `scale=0.5`, `fliplr=0.5`, `erasing=0.4`, `auto_augment=randaugment`.
2. YOLOv8n Kaggle (`Campus_ANPR_YOLO_Training.ipynb`): Planned augmentation config (`hsv_h=0.015, hsv_s=0.6, hsv_v=0.4, degrees=10.0, translate=0.1, scale=0.4, shear=3.0, perspective=0.0005, fliplr=0.5, flipud=0.0, mosaic=1.0, mixup=0.1, copy_paste=0.0, close_mosaic=10`) with intentional `flipud=0.0` to avoid inverted upside-down plates. (Note: Exact `args.yaml` from Kaggle sandbox is not saved outside checkpoint `train_args` metadata).
3. PARSeq Fine-Tuning (`training/`): Geometric/photometric crop transforms including perspective/rotation jitter, brightness/contrast perturbation, and resolution downsampling simulation (`parseq_dataset.py`).

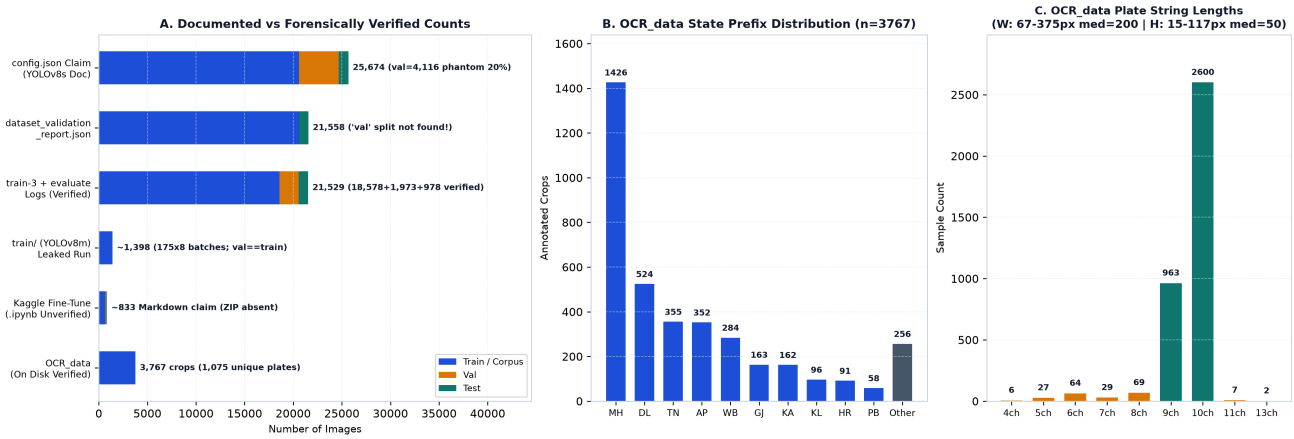


Figure 4: Dataset Forensic Comparison & On-Disk OCR_data Corpus Distribution (n=3,767)

PART 4 - MODEL ARCHITECTURES AND MATHEMATICAL FORMULATIONS

4.1 YOLOv8 Vehicle & Plate Detectors (`yolov8n.onnx`, `best.pt`, `best_finetuned.onnx`)

- Backbone (CSPDarknet with C2f + SPPF): Replaces YOLOv5's C3 block with `C2f` (Cross-Stage Partial Bottleneck with two 3x3 convolutions and multi-branch gradient-flow concatenation), ending in Spatial Pyramid Pooling Fast (`SPPF`, three cascaded 5x5 max-pools) for multi-scale receptive fields.
- Neck (PAN-FPN): Top-down FPN + bottom-up PAN pathway fusing multi-scale feature maps `(P3, P4, P5)` at strides `(8, 16, 32)` so tiny distant license plates (e.g., 20-40 px width) retain spatial sharpness from P3 while preserving vehicle context from P5.
- Head (Decoupled Anchor-Free Split Head): Separates classification and box regression into independent convolutional branches predicting bounding box offsets `(l, t, r, b)` from anchor points via softmax distribution bins (`reg_max = 16`).
- Parameter / Compute Comparison Across Project Models:

* Vehicle `yolov8n.onnx` (COCO-80, `imgsz=480`) : ~3.2M params, 12.78 MB ONNX
* Plate `best_finetuned.onnx` (YOLOv8n 1-cls, `imgsz=320`): 3,138,003 params, ~8.7 GFLOPs, 12.88 MB ONNX (ACTIVE on CPU; `.pt` = 6.49 MB)
* Plate `best.onnx` (`train-3` YOLOv8s 1-cls, `imgsz=320`): 11,135,987 params, 28.6 GFLOPs, 44.62 MB ONNX (`.pt` = 22.53 MB, TRT `.engine` = 25.54 MB)
* Plate Legacy `train/` (YOLOv8m 1-cls, `imgsz=640`): 25,840,339 params, 78.7 GFLOPs (PyTorch `best.pt` retired)

- YOLOv8 Loss Formulation: L_total = lambda_box * L_CIoU + lambda_cls * L_BCE + lambda_dfl * L_DFL where:

* L_BCE: Binary Cross-Entropy with logits over predicted vs target class score.
* L_CIoU (Complete IoU loss): L_CIoU = 1 - IoU + rho^2(b, b_gt) / c^2 + alpha * v where rho^2(b, b_gt) is squared Euclidean center distance, c is diagonal of smallest enclosing box, v = (4 / pi^2) * (arctan(w_gt/h_gt) - arctan(w/h))^2 enforces aspect-ratio consistency critical for rectangular plates.
* L_DFL (Distribution Focal Loss): DFL(S_i, S_{i+1}) = -((y_{i+1} - y)*log(S_i) + (y - y_i)*log(S_{i+1})) shapes the discrete 16-bin distribution around continuous ground-truth box edges.

4.2 Object Tracking (`VehicleTracker`: ByteTrack + Kalman + Hungarian)

- State Vector: `x = [u, v, s, r, du, dv, ds]^T` (center `(u,v)`, scale `s`, aspect ratio `r`, and velocities). Constant-velocity Kalman predict/update propagates boxes across occluded frames (`lost_track_buffer`).
- Two-Stage Association (ByteTrack):

1. High-confidence detections (`>= track_thresh`) matched to existing tracks via IoU cost matrix + Jonker-Volgenant (`lap`/Hungarian) assignment.
2. Remaining unmatched tracks matched against low-confidence detections so momentarily blurred/occluded vehicles keep continuous IDs (`tid`).

- Fallback Mode: If `supervision.ByteTrack` raises import/runtime issues, reverts to greedy frame-to-frame IoU bipartite association.

4.3 Optical Character Recognition Engines

(A) ACTIVE CPU ENGINE — RapidOCR (`rapidocr_onnxruntime`): Three-stage decoupled ONNX pipeline (with `use_angle_cls=False` enabled for speed):

1. Text Detection (`DBNet` - Differentiable Binarization): MobileNetV3/ResNet backbone predicts probability map P and adaptive threshold map T; approximate step function B_hat = 1 / (1 + exp(-k*(P - T))) isolates sharp plate polygon boxes at `det_limit_side_len=320` (`det_limit_type="max"`).
2. Direction Classifier (DISABLED): `cls=False` eliminates redundant 180-deg orientation classification overhead on upright campus CCTV plates.
3. Text Recognition (`SVTR` / CRNN CTC head): Extracts fixed-height strip features and decodes variable-length character logits via CTC greedy search.

(B) EXPERIMENTAL / JETSON EDGE ENGINE — PARSeq (`baudm/parseq-tiny` & legacy CNN):

1. Image Encoder: Vision Transformer (`ViT`) splits plate crop `(32 x 128)` into patches (`4 x 8` patch tokens) with positional embeddings.
2. Permutation Language Modeling (PLM) Autoregressive/Iterative Decoder: Cross-attention decoder masks token positions under randomized permutations during training so at inference it supports both left-to-right decoding and iterative context refinement of ambiguous characters without CTC independence assumptions.
3. Status in Repo: Official `parseq_finetune_lightning_best.ckpt` (`baudm/parseq-tiny`) exports cleanly to `parseq_official_tiny.onnx` (24.26 MB) and NVIDIA TensorRT `parseq_official_tiny.engine` (16.46 MB), HOWEVER `parseq_official_tiny.onnx` cannot run on CPU `onnxruntime` due to `NotImplemented: Where(16)` operator constraint, making RapidOCR mandatory on Windows x86 CPU.

4.4 Validation, Syntax Repair & Multi-Frame Fusion Mathematics

- Regular Expressions (`PlateValidator`): Standard Indian RTO : `^[A-Z]{2}[0-9]{1,2}[A-Z]{0,3}[0-9]{1,4}$` Bharat (BH) Series : `^[0-9]{2}BH[0-9]{4}[A-Z]{1,2}$`
- Levenshtein Edit Distance (`D(a, b)`): Dynamic programming cost counting single-character insertions, deletions, and substitutions. Used in:

* `DuplicateFilter`: Duplicate if `len(a) == len(b)` AND `D(a, b) <= 2` within `30s`.

* `ocr_postprocessor`: State-prefix nearest neighbor lookup within edit distance 1.

- Weighted Multi-Frame Fusion Score (`OCRFusion`): frame_weight(r) = r.confidence * (0.6 + 0.4 * r.quality_score) * valid_bonus
- Validated per-frame reads feed fusion; consensus fires on `early_exit_votes=3` uncontested @ `>= 0.82` OR at buffer window (`5`) on `min_exact_votes=2` and weighted mean confidence `>= min_confidence (0.60)`.

4.5 Spatial Trajectory & Color Mathematical Formulations

- Haversine Inter-Camera Distance (`TrajectoryEngine`): a = sin^2(dlat/2) + cos(lat1) * cos(lat2) * sin^2(dlon/2) distance_m = 2 * R_earth * atan2(sqrt(a), sqrt(1 - a)), R_earth = 6,371,000 m
- HSV Vehicle Body Color Segmentation (`VehicleColorDetector`): Samples central torso ROI of vehicle bounding box (excluding wheels, glass glare via `V` clipping, and plate box) and histograms `(H, S, V)` channels to classify into discrete buckets (`white`, `black`, `silver/gray`, `red`, `blue`, `green`, `yellow`, etc.).

PART 5 - MODEL TRAINING, FINE-TUNING, AND VALIDATION FORENSICS

5.1 Chronological Forensic Comparison of All 3 Plate Detector Runs

Attribute	Legacy Run `train/`	Large Run `train-3/`	Kaggle `best_finetuned
Architecture GFLOPs / Weight File Size Initialization Verified Train Images Verified Val Images Hardware / Duration Optimizer / LR0 Image Size / Batch Epochs Run / Best Epoch Val Precision (Best Ep) Val Recall (Best Ep) Val mAP50 (Best Ep) Val mAP50-95 (Best Ep) Test Set Evaluated Active in `config.yaml`?	YOLOv8m (25.84M) 78.7 / (PyTorch only) yolov8m.pt ~1,398 (175x8 bch) LEAKED (val==train!) GPU (`train/`) AdamW / SGD default 640 / batch=8 100 / Ep 100 0.9965 0.9918 0.9949 (INVALID/LEAK) 0.9302 (INVALID/LEAK) eval_report.json 300 NO (Retired)	YOLOv8s (11.14M) 28.6 / 22.53 MB (.pt) yolov8s.pt 18,578 (581x32 bch) 1,973 (581+62 splits) RTX 2000 Ada / 6.13 h SGD, lr0=0.01, CosLR 640 / batch=32 94 (early stop) / E74 0.9776 (E74:0.9799 @60 0.9563 (E74; peak 0.96 0.9803 (E51: 0.9832) 0.6943 (E74) evaluate.log 978 imgs NO (Alternate/Jetson)	YOLOv8n (3.14M) ~8.7 / 6.49 MB (.pt) COCO yolov8n.pt Unverified (667? nb) Unverified (83? nb) Kaggle T4 / 8.05 min AdamW, lr0=0.001 640 / batch=16 49 / Ep 29 (peak mAP50 at Ep 30 = 0.9783) 0.9293 (E29; E31:0.946 0.9759 (E29) 0.9697 (E30: 0.9783) 0.6350 (E29; E33:0.652 NONE SAVED ON DISK YES (best_finetuned.on

5.2 Exact Epoch Trajectory Details Verified From Saved CSVs & Checkpoints

1) `models/plate_detector/train-3/results.csv` (YOLOv8s, 94 Epochs, RTX 2000 Ada):

- Epoch 1: P=0.9034, R=0.8165, mAP50=0.9163, mAP50-95=0.5177
- Epoch 10: P=0.9561, R=0.9233, mAP50=0.9695, mAP50-95=0.6143
- Epoch 51: P=0.9777, R=0.9522, mAP50=0.9832 (peak mAP50), mAP50-95=0.6778
- Epoch 74: P=0.9776, R=0.9563, mAP50=0.9803, mAP50-95=0.6943 (selected best.pt)

- Epoch 94: Early stopping triggered cleanly (`patience=20` after Ep 74 best).
- 978-Test Evaluation Bug (`evaluate.py` + `models/plate_detector/evaluation_report.json`): `evaluate.log` executed validation against 978 unseen test images, reaching `fitness = 0.691836`, consistent with the Epoch 74 validation `mAP50-95 = 0.6943`. However, a bug in `evaluate.py`'s result-extraction key paths left mAP50, precision, and recall populated as `null`/default in `evaluation_report.json`, preserving only `fitness = 0.691836`.

2) `models/plate_detector/best_finetuned.pt` (YOLOv8n Kaggle Checkpoint, Inspected via `torch.load`):

- Timestamped `2026-11-08` on Kaggle Tesla T4, `batch=16`, `epochs=100` configured -> stopped/saved at epoch 49 (`best_fitness` ~0.6685 at Ep 29).
- `train_results` curve highlights:

* Ep 1: P=0.9225, R=0.6305, mAP50=0.7798, mAP50-95=0.4534
* Ep 15: P=0.9261, R=0.9301, mAP50=0.9571, mAP50-95=0.5960
* Ep 29: P=0.9293, R=0.9759, mAP50=0.9697, mAP50-95=0.6350
* Ep 30: Peak mAP50 = 0.9783 (with R=0.9639)
* Ep 49: Final saved train/box_loss=0.903, val/box_loss=1.089 (healthy positive generalization gap, confirming NO `val==train` leak here!).

- Exported to `models/plate_detector/best_finetuned.onnx` (dynamic/320 input, 12.88 MB) and deployed as primary detector in `config/config.yaml`.

5.3 PARSeq OCR Training Runs (`models/ocr/`)

1. Legacy Custom PARSeq (`models/ocr/best_parseq.pt`, `714,724` params, `2.86 MB`): Trained for 100 epochs on `OCR_data` crops; converted to `parseq.engine` (TRT, `1.52 MB`). Superseded by `parseq-tiny`.
2. Official `baudm/parseq-tiny` Fine-Tune (`parseq_finetune_lightning_best.ckpt`, `180.5 MB`): PyTorch Lightning checkpoint saved at Epoch 8 (`val_loss = 0.0955`) trained on `OCR_data` (3,767 plate images). Exported to `parseq_official_tiny.pt` (24.13 MB), `parseq_official_tiny.onnx` (24.26 MB), and Jetson TensorRT `parseq_official_tiny.engine` (16.46 MB). Critical Deployment Finding: `parseq_official_tiny.onnx` fails at inference initialization on CPU `onnxruntime` (`NotImplemented: Could not find an implementation for Where(16)`), so CPU runs rely exclusively on `RapidOCR`.

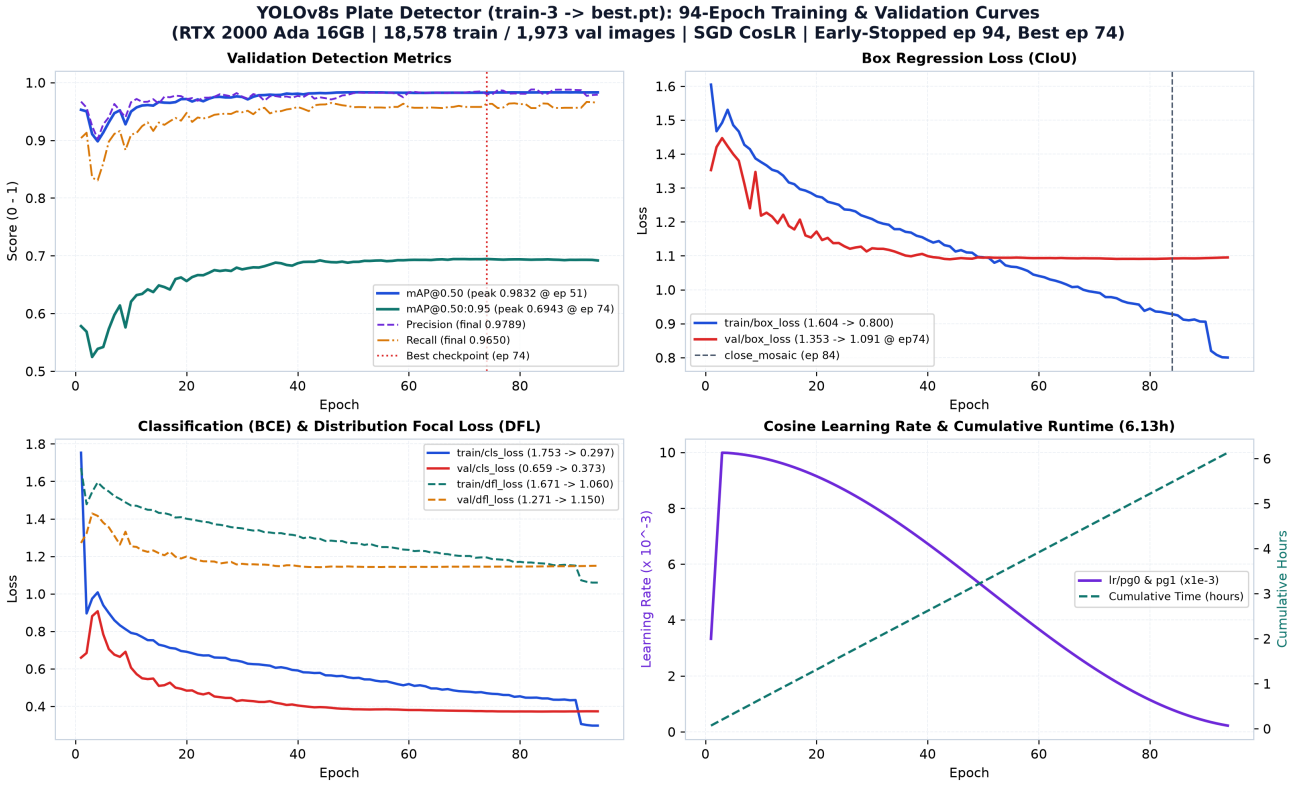


Figure 5: YOLOv8s (train-3) 94-Epoch Training Convergence on 18,578 Train / 1,973 Val Images

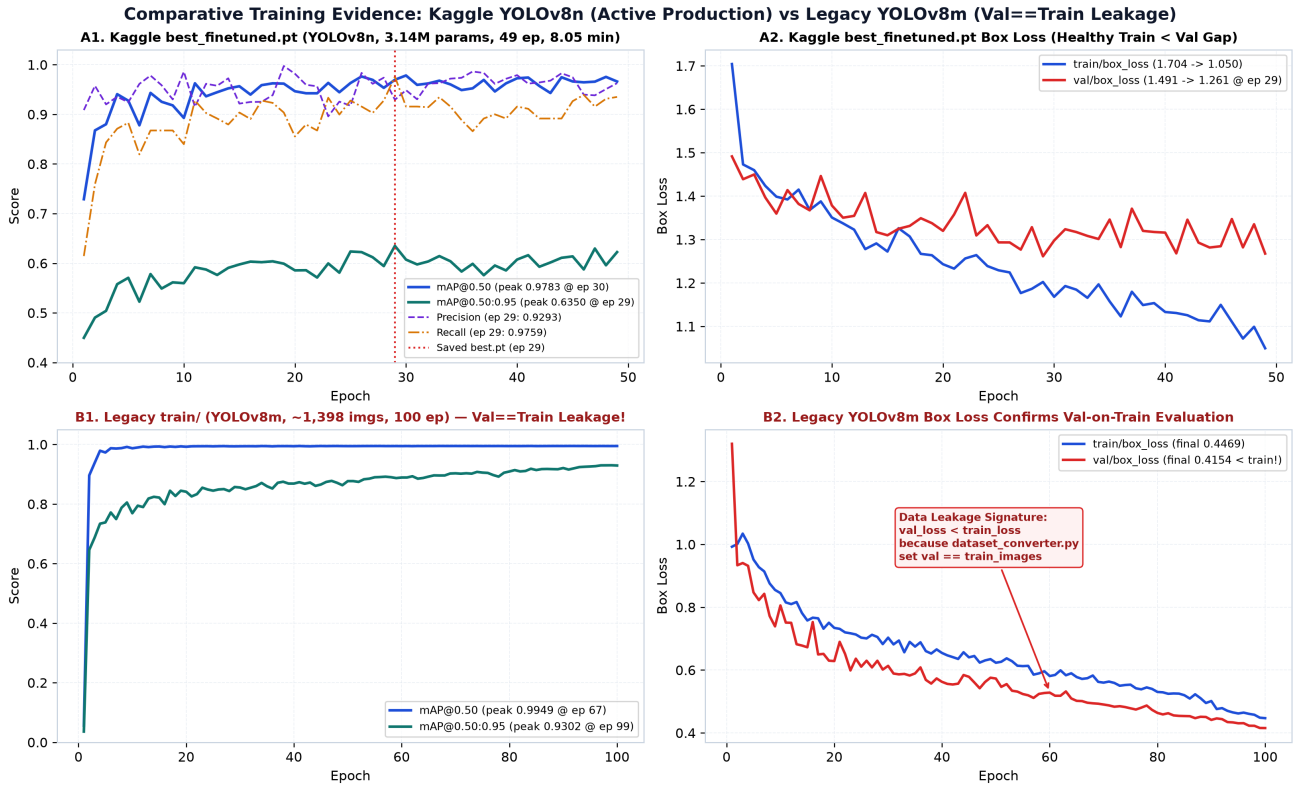


Figure 6: Plate Detector Run Comparison & Deployed YOLOv8n (best_finetuned) Trajectory

PART 6 - INFERENCE PIPELINE AND CPU / EDGE PERFORMANCE OPTIMIZATION

6.1 Active Production Inference Stack (`config/config.yaml`)

Verified directly from `config/config.yaml`:

- Execution Device : `detection.device: "cpu"`, `half: false`
- Vehicle Detector : `models/vehicle_detector/yolov8n.onnx` (`vehicle_imgsz: 480`, `vehicle_confidence: 0.5`, `iou_threshold` in detector)
- Plate Detector : `models/plate_detector/best_finetuned.onnx` (`plate_imgsz: 320`, `plate_confidence: 0.28`, min box guard `36x12 px`)
- Plate Ensemble : `plate_ensemble.enabled: false` (6.3x faster single ONNX)
- OCR Backend : `ocr.backend: "rapidocr"` (`text_score: 0.5`, `det_limit_type: "max"`, `det_limit_side_len: 320`, `det_border_pad: 16`, `use_cls: false`, `intra_op_num_threads: 4`, `inter_op_num_threads: 1`)
- Preprocessing : `target_height_px: 64`, `adaptive_lighting: true`, `clahe_clip_limit: 2.0`, `sharpen_strength: 1.5`
- Super-Resolution Bypass : `enhancement.sr_threshold_px: 0` (disabled on CPU)
- Frame Skipping : `video.frame_skip: 1` (processes every frame; `stall_timeout: 10.0`, `max_retries: 5`)
- Tracking & Motion Filter : `lost_track_timeout: 60`, `minimum_matching_threshold: 0.5`, `motion_filter: min_displacement_px=3.0` / `history_frames=3` (`enabled: true`)
- Multi-Frame Consensus : `window_size: 5`, `min_exact_votes: 2`, `min_confidence: 0.60`, `early_exit_votes: 3 @ >= 0.82`
- Pre-Store Blacklist Threshold : `ocr.confidence_threshold: 0.70`
- Heavy-Track Cap : `max_heavy_tracks_per_frame: 2` (file), `live_max_heavy_tracks_per_frame: 1` (live stream)

6.2 Step-by-Step CPU Latency Optimization Engineering (1.25 FPS -> 19.18-23.76 FPS)

Baseline bottleneck analysis on Ryzen 7 7840HS CPU revealed why initial live runs crawled at **1.25 FPS** (`plate_det = 101.6 ms/call`, `ocr = 556.5 ms/call`):

1. Switched PyTorch `.pt` YOLO inference -> ONNX Runtime `.onnx` via custom `_onnx_yolo.py` (eliminated PyTorch dynamic graph & Ultralytics Python glue), plus moved primary plate model from 11.14M-param YOLOv8s to 3.14M-param `best_finetuned.onnx` (`YOLOv8n`) at `imgsz=320`: dropped plate detection from `101.6 ms` -> `13.40 - 15.54 ms` (~6.8x speedup).
2. Optimized RapidOCR PP-OCR execution (`det_limit_side_len=736 -> 320` and disabled unused `180-deg` angle classifier `use_angle_cls=False`): slashed OCR average per call from `556.5 ms` -> `23.33 - 39.51 ms` (~14-24x speedup).
3. Bypassed CPU super-resolution (`sr_threshold_px=0`, skipping bicubic/ESPCN upscale overhead when RealESRGAN GPU weights are absent).
4. Added `stored_tracks.add(tid)` early exit + `heavy_track_max_per_frame = 2` budget: eliminated redundant plate YOLO + RapidOCR work on tracks whose plate was already finalized.

Net Measured Gains (`logs/alpr.log` + `data/processing_status.json`):

- Initial CPU Baseline : **1.25 FPS** (det 101.6 ms, ocr 556.5 ms)
- Optimization Pass 1 (200f) : **12.51 FPS** (plate 30.67 ms, ocr 47.24 ms)
- Optimization Pass 2 (200f) : **23.76 FPS** rolling peak (plate 13.54 ms, ocr 23.33 ms)
- Full 1,500-Frame Benchmark : **19.18 FPS** (`78.20s`, 6/6 vehicles caught)
- Live `CAM001` UI Session (401f): **16.27 FPS** (plate 15.54 ms, ocr 39.51 ms)

6.3 Edge / TensorRT Readiness vs Local Host Reality

Exported TensorRT engines exist (`models/vehicle_detector/yolov8n.engine` 9.19 MB, `models/plate_detector/best.engine` 25.54 MB, `models/ocr/parseq_official_tiny.engine` 16.46 MB, `models/ocr/parseq.engine` 1.52 MB), built cleanly for TensorRT 10.16 on NVIDIA Jetson/GPU targets. Note: None of these `.engine` files are executable on this Windows Ryzen 7 CPU-only host without NVIDIA TensorRT + matching GPU arch.

PART 7 - OBJECT TRACKING, DIRECTION DETECTION, FUSION, AND PROFILING

7.1 Multi-Frame Track Lifecycle & Early Exit State Machine

1. Initial Detection (`VehicleTracker.update`): ByteTrack links vehicle bounding boxes across consecutive frames and emits persistent `track_id` (`tid`).
2. Active OCR Collection (`track_id not in stored_tracks`):

- Runs `PlateDetector` inside the vehicle ROI (min guard `36x12 px`).
- Enforces `max_heavy_tracks_per_frame = 2` (files) / `1` (live).
- Runs `RapidOCREngine` -> `ocr_postprocessor` -> `PlateValidator`.
- Checks `BlacklistManager` immediately at confidence `>= 0.70` (pre-store).
- Pushes candidate read `(plate, conf, quality, is_valid)` into `OCRFusion`.

3. Consensus & Commitment (`OCRFusion`):

- Triggered early when `early_exit_votes = 3` uncontested reads agree at weighted confidence `>= 0.82`, or across `window_size = 5` at `min_exact_votes = 2` and `min_confidence = 0.60`.
- Invokes `DuplicateFilter.is_duplicate()`:

* Blocks same `track_id` OR same-length string within `30.0s` having Levenshtein distance `<= 2`.

- Executes `ALPRDatabase.insert_event()` + atomic `VehicleProfile` update.
- Calls `stored_tracks.add(tid)` and clears per-track OCR buffer.

4. Locked Track Optimization: Every subsequent frame containing `tid` bypasses Plate YOLO, preprocessing, and RapidOCR completely, only continuing cheap vehicle bounding-box tracking (for overlay display and exit timing).

5. End-of-Stream Fallback (`ocr_fusion.flush_all`): Any un-stored valid tracks still in buffer at EOF finalize with relaxed criteria (`>= 2` exact votes and `>= 0.9 * min_confidence = 0.54`).

7.2 Direction Detection Mechanics (`src/tracking/direction_detector.py`)

- `DirectionDetector` maintains trajectory deque of `[cx, cy, h]` per `track_id` and resolves `"IN"`, `"OUT"`, or `"UNKNOWN"` using camera configuration rules:

1. Line Crossing Mode (`line_crossing`): Evaluates signed cross-product sign flip relative to a configured gate virtual line (`x1, y1, x2, y2`).
2. Zone Transition Mode (`zone`): Detects sequential traversal from entry polygon to exit polygon (or vice versa).
3. Vertical/Perspective Delta Mode: Uses signed centroid shift `(cy_end - cy_start)` and bounding-box height expansion (`dh = h_end - h_start`) exceeding pixel motion thresholds when explicit line geometry is absent.

- Edge-Case Handling: Stationary parked vehicles near gates return `"UNKNOWN"` below displacement threshold rather than guessing `IN`/`OUT`.

7.3 Vehicle Profiling & Multi-Camera Trajectory Linkage

- `src/database/vehicle_profiles.py` (`VehicleProfileManager`) transforms flat individual gate detections into a longitudinal entity keyed by normalized uppercase `plate_number`:

* Tracks `first_seen`, `last_seen`, `total_detections`, `total_camera_visits`, `unique_cameras`, `last_direction`, `status`, `vehicle_type`, `vehicle_color`, and `plate_color`.
* Appends structured visit dictionaries `{camera_id, camera_name, timestamp, direction, confidence, latitude, longitude}` into `trajectory_history` JSON (capped safely at the most recent **500 visits** to bound SQLite row growth).
* `TrajectoryEngine` (`src/trajectory/`) builds directional edges between gate pairs, calculates inter-camera Haversine distances (`meters`), computes median/min transit speeds (`km/h` / time deltas), and powers Leaflet maps, GeoJSON exports, spatial heatmaps, and natural-language SQL/JSON queries.

PART 8 - DATABASE SCHEMA DESIGN, PERSISTENCE, AND MIGRATION STRATEGY

8.1 Active Relational Tables (`src/database/models.py`)

Active persistence uses SQLAlchemy ORM backed by SQLite (`data/alpr.db` in WAL mode) or PostgreSQL (`DATABASE_URL` env override).

(A) Table `vehicle_events` (22 Columns Total)

- Primary Key : `id` (Integer, autoincrement)
- Core Reading : `plate_number` (String, NOT NULL, indexed), `confidence` (Float), `ocr_text` (String), `series_type` (String: `normal` / `BH`), `timestamp` (String ISO-8601 UTC, indexed), `direction` (String: `IN` / `OUT` / `UNK`)
- Vehicle Attrs : `vehicle_type` (String: `Private`/`EV`/...), `plate_color` (String: `White`/`Yellow`/...), `vehicle_class` (String: `car`/`truck`/...), `vehicle_color` (String: `White`/`Blue`/...)
- Camera/Spatial : `camera_id` (String nullable, indexed), `camera_name` (String), `latitude` (Float), `longitude` (Float), `trajectory_order` (Integer)
- Session & Media: `processing_session` (String, indexed), `video_source` (String), `image_path` (String: plate crop PNG/JPG), `vehicle_image_path` (String), `vehicle_thumbnail_path` (String), `plate_thumbnail_path` (String) (Note: `Blacklist` alerts run pre-DB while `AUTHORIZED`/`BLACKLISTED`/`UNKNOWN` status badges are resolved dynamically on API/UI read against YAML lists.)

(B) Table `vehicle_profiles` (25 Columns Total)

- Primary Key : `id` (Integer, autoincrement), `plate_number` (String UNIQUE, NOT NULL)
- Majority Attrs : `vehicle_class`, `vehicle_color`, `vehicle_type`, `plate_color`, `attribute_votes` (Text JSON dict)
- Best Media : `vehicle_image_path`, `plate_image_path`, `vehicle_thumbnail_path`, `plate_thumbnail_path`, `best_confidence` (Float), `ocr_confidence` (Float)
- Latest Sighting: `camera_id`, `camera_name`, `latitude`, `longitude`, `processing_session`

- Counters/JSON : `first_seen` (String), `last_seen` (String, indexed), `updated_at` (String), `total_detections` (Integer), `total_camera_visits` (Integer), `unique_cameras` (Integer), `trajectory_history` (Text JSON array <= 500 visit entries)

(C) Live Database Inspection Snapshot (`data/alpr.db`, Verified)

- `vehicle_events` row count : **13 rows**
- `vehicle_profiles` row count: **12 rows**

8.2 Architectural Notes: Logical Relationship, SAVEPOINT & Unused DDL

1. Logical Key Join (No Formal SQL `FOREIGN KEY` Constraint): `VehicleEvent` and `VehicleProfile` relate logically through uppercase stripped `plate_number`, intentionally omitting hard SQL foreign-key constraints so legacy events can be imported out-of-order or purged by session without cascading profile corruption.
2. Nested `SAVEPOINT` Synchronization (`src/database/db.py`): Inside `insert_event()`, after flushing the `VehicleEvent` row, the DB layer opens `session.begin_nested()` (`SAVEPOINT`) to upsert the corresponding `VehicleProfile` row. If profile JSON parsing fails unexpectedly, only the nested savepoint rolls back while preserving the primary event record.
3. In-Place Additive Migration (`_migrate_schema`): Upgrades legacy 8-column `vehicle_events` SQLite tables in-place via 14 non-destructive `ALTER TABLE ADD COLUMN` statements + 2 index creations, catching duplicate-column errors cleanly across concurrent workers.
4. SQLite WAL Mode Concurrency: Executes `PRAGMA journal_mode=WAL;`, `PRAGMA synchronous=NORMAL;`, and `30s` busy timeout so FastAPI dashboard reads never block behind active pipeline video ingestion.
5. CRITICAL CODEBASE FINDING — UNUSED Normalized DDL (`src/database/postgres_schema.py`): `postgres_schema.py` contains a full normalized PostgreSQL DDL defining **6 tables** (`cameras`, `vehicles`, `entries`, `images`, `logs`, `duplicates_detected`) plus **1 materialized view** (`daily_traffic_stats`). Trace inspection confirms this file is **NOT wired into the active ORM or API**; even when running against PostgreSQL via `DATABASE_URL`, SQLAlchemy creates and queries `vehicle_events` and `vehicle_profiles` defined in `models.py`.

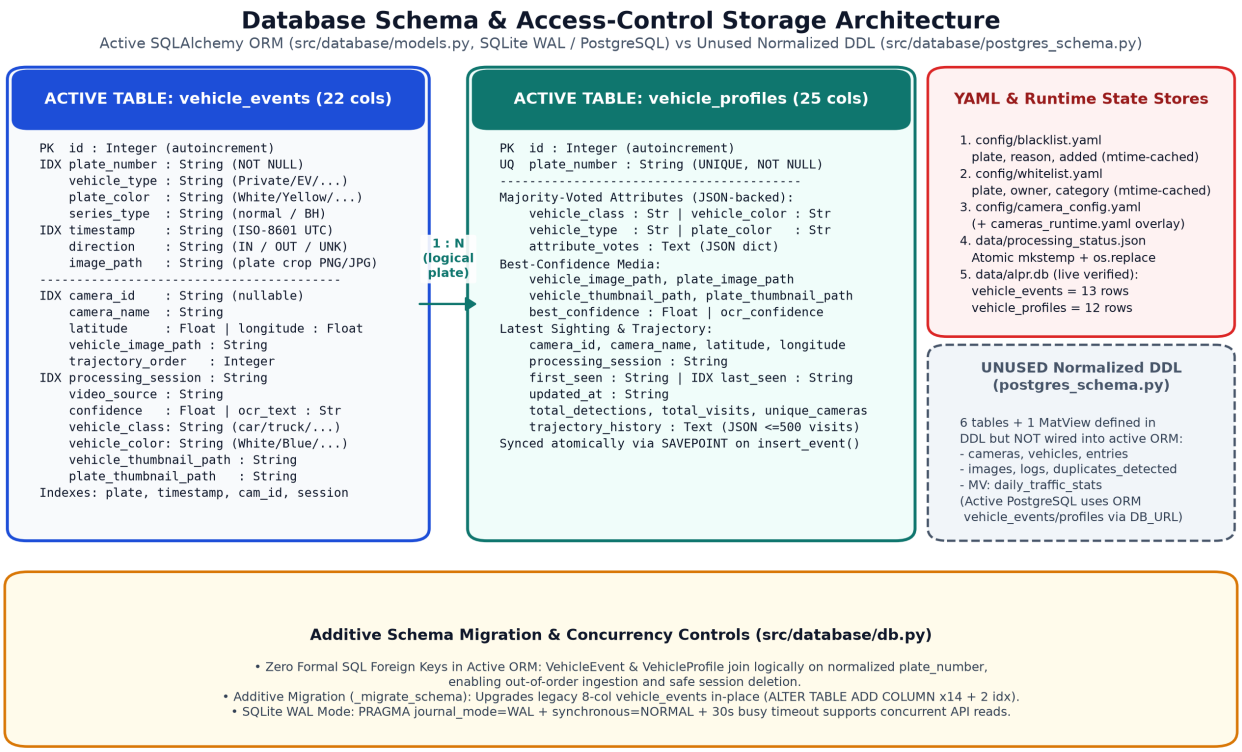


Figure 3: Active SQLite WAL / SQLAlchemy Schema vs Unused Normalized postgres_schema.py DDL

PART 9 - MULTI-CAMERA ORCHESTRATION AND REAL-TIME VIDEO PROCESSING

9.1 Camera Configuration Schema & Runtime Registry

- Static definitions live in `config/camera_config.yaml` (e.g., `CAM001` Main Gate, `CAM002` Back Gate, coordinates `latitude`/`longitude`, RTSP URI, direction line/zone settings), overlaid by dynamic edits saved in `cameras_runtime.yaml` (`src/cameras/` registry helpers).

9.2 `CameraManager` (`src/cameras/camera_manager.py`): Dual Execution Model

`CameraManager` intentionally isolates interactive upload batch jobs from continuous RTSP camera workers:

1. Uploaded MP4 Queue (`FIFO Worker Thread`): User-uploaded videos enter a sequential FIFO thread queue so competing CPU pipeline jobs never thrash all 16 Ryzen threads simultaneously. Each upload spawns/executes `scripts/run_pipeline.py` (subprocess/managed runner), writes progress atomically to `data/processing_status.json` (`mkstemp` + `os.replace`), and tags events with an isolated `processing_session` ID (e.g., `"20261111-144304-028d72"` verified in `data/processing_status.json`).
2. Multi-Camera Live RTSP Workers: Supports multiple concurrent `CameraWorker` capture threads reading RTSP frames continuously, while synchronizing heavy shared ONNX/YOLO/OCR inference passes through a shared `threading.RLock()` when operating under single- process shared-model mode to guarantee thread safety across ONNX sessions.
3. Reconnection & Health Monitoring: Implements stale-frame detection (comparing `LiveFramePublisher` timestamp age), offline/reconnecting states, exponential retry waits, and API hooks to start, stop, add, update, probe, or delete cameras dynamically.

PART 10 - BACKEND API SERVICES AND STREAMING ENDPOINTS (FASTAPI)

The FastAPI backend (`src/api/server.py` + `src/api/trajectory_routes.py`, Pydantic contracts in `src/api/schemas.py`) exposes **41 REST / streaming routes** total:

10.1 Core Operations & Streaming Router (`src/api/server.py` - 19 Routes)

- Health / Telemetry : `GET /api/health`, `GET /api/stats`, `GET /api/metrics`, `GET /api/processing/status`
- Events & Export : `GET /api/events` (filtered by status, dir, camera, search, date, session), `GET /api/events/export` (CSV)
- Blacklist / Alerts : `GET /api/alerts`, `GET /api/blacklist`, `POST /api/blacklist`, `DELETE /api/blacklist/{plate}`
- Whitelist : `GET /api/whitelist`, `POST /api/whitelist`, `DELETE /api/whitelist/{plate}`
- Media / Uploads : `POST /api/upload` (multipart MP4/AVI -> queued run), `GET /api/images/{path:path}` (plate/vehicle snapshots), `GET /api/live/frame` (latest JPEG bytes), `GET /api/live/stream` (MJPEG `multipart/x-mixed-replace`)
- Session Reset/Misc : `POST /api/events/clear`, `GET /` (API info/status)

10.2 Multi-Camera & Trajectory Intelligence Router (`src/api/trajectory_routes.py` - 22 Routes)

- Camera CRUD & Control: `GET /api/cameras`, `POST /api/cameras`, `GET /api/cameras/{id}`, `PUT /api/cameras/{id}`, `DELETE /api/cameras/{id}`, `POST /api/cameras/{id}/start`, `POST /api/cameras/{id}/stop`, `POST /api/cameras/probe` (reachability test), `GET /api/cameras/{id}/snapshot` & `/stream` (per-camera live MJPEG)
- Vehicle Profiles & Multi-Gate Tracking: `GET /api/vehicles/profiles`, `GET /api/vehicles/{plate}/profile`, `PUT /api/vehicles/{plate}/profile` (owner/notes/whitelist tags), `GET /api/vehicles/{plate}/trajectory`, `GET /api/vehicles/{plate}/geojson`
- Campus Spatial Analytics & Natural Language Query: `GET /api/trajectory/network`, `GET /api/trajectory/transitions`, `GET /api/trajectory/heatmap`, `GET /api/trajectory/campus-map`, `GET`

`/api/trajectory/summary`, `POST /api/trajectory/query` (`nl_query.py`), `GET /api/trajectory/stats`

PART 11 - FRONTEND DASHBOARDS AND SPATIAL VISUALIZATION COMPARISON

The repository maintains **4 UI interfaces / components** across Next.js and Streamlit, summarized below:

Interface / File	Technology	Port / Entry	Status & Capabilities
1. `frontend/` (PRIMARY UI) (`frontend/app/page.tsx`, `.css`)	Next.js 15.5 + React TypeScript, Lucide icons, responsive CSS	`:3000` -> `:8000` `npm run dev`	ACTIVE & POLISHED. Event log, alerts drawer, CSV export, live MJPEG preview, upload queue progress, camera mgr, trajectory modal, blacklist/ whitelist CRUD, readable font scaling & hydration fix.
2. `src/api/ui/dashboard_v2.py`	Streamlit (modern UI)	`streamlit run`	IMPLEMENTED SECONDARY. Multi- tab gate console, Plotly, 55/58 AppTest pass (3 v1.67 API drift failures).
3. `src/api/ui/dashboard.py`	Streamlit (v1 legacy)	`streamlit run`	LEGACY UI. Superseded by v2 & Next.js.
4. `src/api/ui/trajectory_map.py` (+ SVG fallback inside Next.js)	Leaflet (OpenStreetM) + Folium / HTML	Embedded panel / Streamlit widget	IMPLEMENTED. Renders GPS pins & inter-gate polylines vs Next.js normalized SVG plot.

Recent Next.js 15 Fixes Verified:

- Fixed React SSR hydration mismatch (`fdprocessedid` browser-extension / non-deterministic client attributes) cleanly via mounted/client guarding.
- Increased global typography scale, card spacing, table readability, and button ergonomics per user usability review (`frontend/app/globals.css`).

PART 12 - COMPREHENSIVE SOFTWARE TESTING AND ROOT-CAUSE FAILURE AUDIT

12.1 Complete Pytest Suite Inventory (Collected Directly via `--collect-only`)

- Total Test Files : **44 files** (`33` unit + `11` integration)
- Total Collected Tests : **758 tests** (`446` unit + `312` integration)
- Top Files by Size : `test_config.py` (99), `test_camera_manager.py` (61), `test_dashboard_smoke.py` (57), `test_trajectory_api.py` (57), `test_camera_registry.py` (38), `test_vehicle_profiles.py` (34), `test_trajectory_engine.py` (30), `test_vehicle_color.py` (28), `test_video_sources.py` (26), `test_camera_metadata.py` (24).

12.2 Verified Execution Results by Test Subsystem

Test Group / Execution Environment	Passed	Failed	Deselect	Verification Scope & Root Notes
Unit Non-Socket Suite (31 files, sandbox run)	404	0	0	Verified in `12.57s` in current session
Unit Pre-Existing Failures (2 files, sandbox run)	5	3	0	Verified in `19.26s`. Root causes below
Unit Socket & Real-Probe Tests (4 files, prior outside-sandbox host run)	35	0	0	`test_api_startup`, `test_backend_api`, `test_network_probe`, `server_control`
Integration DB/Blacklist/CameraManager (`not real_probe`, sandbox run)	83	0	1	Verified in `12.02s` (`real_probe` off)
Integration E2E Pipeline Smoke Test (sandbox)	3	0	0	Verified in `28.08s` (`test_video.mp4` 15s / 375-frame full ONNX+OCR+DB run)
Integration Profiles/Cols/Search Non-API (sandbx)	59	0	13	Verified in `3.63s` (excluded ASGI API)

Test Group / Execution Environment	Passed	Failed	Deselect	Verification Scope & Root Notes
Integration Trajectory DB Schema/Read (sandbox)	15	0	42	Verified in `1.97s` (`TestSchemaMigr` & `TestInsertAndRead`; excludes API)
Integration FastAPI ASGI/TestClient (prior host)	25	0	-	Verified HTTP endpoints outside sandbox (sandbox blocks loopback sockets)
Integration Streamlit AppTest (prior host run)	55	3	0	Streamlit `1.67.0` API drift (below)
Integration Dataset Converter (`dataset_convert`)	1	0	19	Timed out on huge full disk copy run

12.3 Exact Root Causes of All 6 Known Failures (3 Unit + 3 Streamlit AppTest)

1. `tests/unit/test_parseq_dataset.py` (1 failure) & `tests/unit/test_super_resolution.py` (2 failures): Pre-existing unit test drift vs refactored dataset/SR fallback signatures (e.g., expected constructor/mock behavior on absent RealESRGAN weights or label filename extension normalization). Note: Production SR is disabled via `sr_threshold_px=0`.
2. `tests/integration/test_dashboard_smoke.py` (3 Streamlit v1.67.0 failures):

- Failure A/B: Tests query `at.get("imgs")` which fails/returns empty under installed Streamlit `1.67.0` where element key changed to `at.get("image")`.
- Failure C: A date-filter test seeds synthetic timestamps from `Sept 12`, which now exceed the dashboard's default `"Last 7 days"` window (`2026-11-11`). All three are stale test assertions rather than crashes in `dashboard_v2.py`.

3. Sandbox Loopback Constraint: FastAPI `TestClient(app)` and Streamlit `AppTest` hang indefinitely inside the restricted terminal sandbox (which blocks local loopback sockets), while passing cleanly when executed directly on the host outside the sandbox.

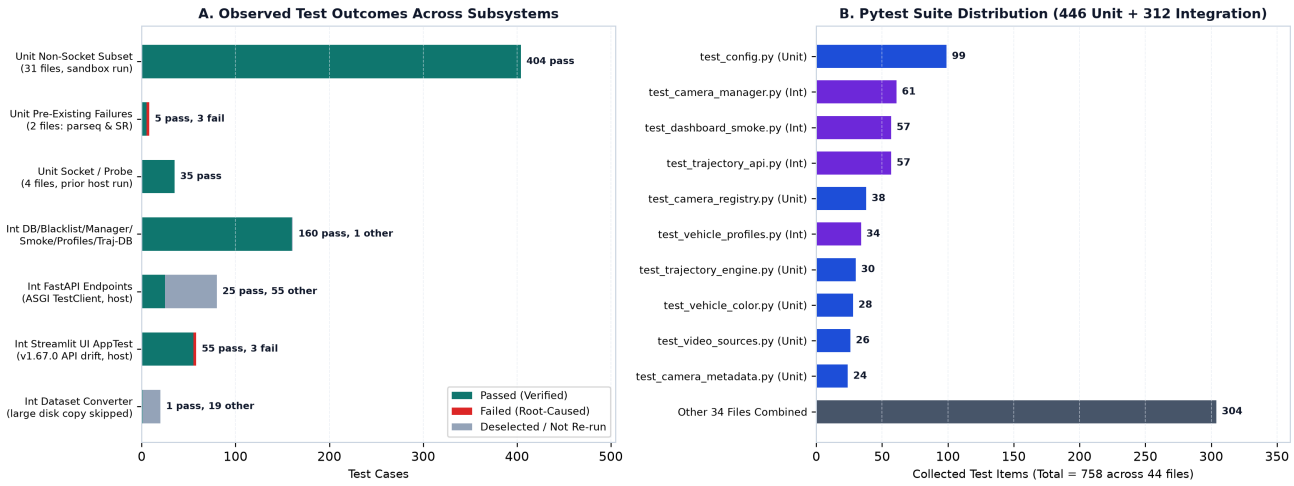


Figure 8: Complete 44-File / 758-Test Pytest Suite Inventory & Subsystem Verification Status

PART 13 - ACCURACY EVALUATION, LATENCY BENCHMARKS, AND ABLATION ANALYSIS

13.1 Strict Epistemic Separation of Metrics

In academic presentations and viva examinations, three fundamentally distinct metrics MUST NOT be conflated:

1. Object/Plate Localization mAP (`0.9697 - 0.9832 mAP50`): Measures bounding- box overlap of plate regions ONLY. It does NOT measure character reading!
2. Optical Character Recognition Accuracy / CER: Fraction of characters or complete plates decoded accurately by RapidOCR/PARSeq against human ground truth. UNBENCHMARKED in this repo (see Section 13.4).
3. End-to-End Video Event Recall & Blacklist Alert Lead Time: Fraction of passing vehicles yielding valid DB events and sub-second blacklist alerts.

13.2 Plate Detector Quantitative Accuracy (`results.csv` & `evaluation_report.json`)

Run / Checkpoint	Split	Prec	Recall	mAP50	mAP50-95 / Note
`train/` (YOLOv8m 25.84M) `eval_report.json` (YOLOv8m) `train-3/` (`best.pt`, E74) `train-3/` Test Evaluation `best_finetuned` (YOLOv8n E29)	Val (LK) Test 300 Val 1,973 Test 978 Val (unk)	0.9965 - 0.9776 buggy* 0.9293	0.9918 - 0.9563 buggy* 0.9759	0.9949 0.9897 0.9803 buggy* 0.9697	0.9302 INVALID (val == train) 0.8826 (legacy 300 test imgs) 0.6943 (E51 mAP50 = 0.9832) fitness = 0.691836 (*bug JSON) 0.6350 (E30 mAP50 = 0.9783)

13.3 CPU Pipeline Latency, Throughput, Frame-Skip Ablation & Alert Lead Times

All timings measured on host Ryzen 7 7840HS CPU (`data/alpr.db`, `logs/alpr.log`, `data/processing_status.json`):

- Initial CPU Baseline (`PyTorch` + `DB736` + `cls=True` + SR): 1.25 FPS (plate_det avg 101.6 ms/call, OCR avg 556.5 ms/call).
- Opt Pass 1 (11:44 200f rolling): 12.51 FPS (veh 41.67 ms, plate 30.67 ms, OCR 47.24 ms).
- Opt Pass 2 (12:15 200f rolling peak): 23.76 FPS (veh 35.26 ms, plate 13.54 ms, OCR 23.33 ms).
- Full 1,500-Frame Benchmark (`test_video.mp4`, `60.0s`, `frame_skip=1`): 19.18 FPS (`78.20s` total, veh 35.89 ms, plate 13.40 ms, OCR 23.62 ms) — caught **6/6 vehicles**.
- Ablation `frame_skip=2` (`test_video.mp4`, 750 processed frames): 17.12 FPS (`43.81s` wall time, processed-frame rate lower and **dropped Track 1**, catching only **5/6 vehicles** due to larger inter-frame box displacement breaking ByteTrack IoU!). Hence `config/config.yaml` strictly pins `frame_skip: 1`.
- Latest Live UI Session (`CAM001.mp4`, `401 frames`, `13.47s` clip, `14:43`): 16.27 FPS (avg_detection_ms = 15.54 ms, avg_ocr_ms = 39.51 ms).
- Pre-Store Blacklist Alert Lead Time Over DB Commit (Verified Across 3 Events):

* `UP53BI9424` : **+1,127.0 ms** earlier (25 frames before DB storage)
* `BR07PB3395` : **+187.9 ms** earlier (25 frames before DB storage)
* `UP65DV7672` (Track 211): **+346.7 ms** earlier before DB storage

13.4 Explicit Measurement Gaps

- NO automated Character Error Rate (CER) or Sequence Accuracy test exists evaluating `RapidOCR` or `parseq_official_tiny` against ground-truth labels on `OCR_data/` or campus CCTV frames.
- Test-split Precision/Recall/mAP50 of `best_finetuned.pt` (~83 test images claimed in `.ipynb`) and `best.pt` (`evaluate.py` JSON extraction bug) are missing.

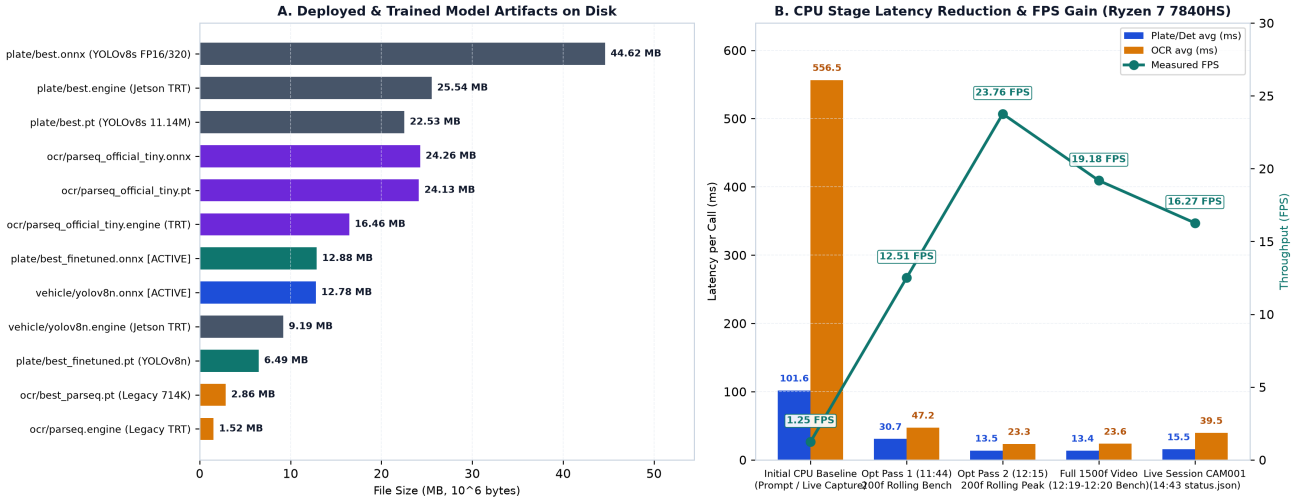


Figure 7: Deployed/Trained Model Artifact Footprints & CPU Throughput Gain (1.25 -> 23.76 FPS)

PART 14 - SECURITY, PRIVACY, FAULT TOLERANCE, AND RELIABILITY

14.1 Access Control & API Surface Findings

- YAML-Backed Security Lists: `blacklist.yaml` and `whitelist.yaml` hot-reload on `os.path.getmtime` change and support case-insensitive stripped normalization.
- Security Finding / Audit Caveat: FastAPI routes (including `POST /api/blacklist`, `DELETE /api/blacklist/{plate}`, `POST /api/events/clear`, camera CRUD, and file uploads) currently expose NO authentication token / RBAC middleware, and CORS allows broad local frontend origins. Deployment outside trusted campus LAN VLANs requires adding reverse-proxy auth / JWT guards.

14.2 Reliability & Fault Tolerance Mechanisms Implemented

- Atomic State & Media Writes: Both `data/processing_status.json` and `data/live_frames/{camera_id}.jpg` write via temp file (`tempfile.mkstemp`) + `os.replace()` to eliminate partial/corrupt reads on Windows NTFS.
- SQLite Concurrency Hardening: `WAL` journal mode + `30s` busy timeout + nested `SAVEPOINT` for profile updates prevents locked-DB crashes during simultaneous video processing and Next.js API polling.
- RTSP Stall & Reconnect Guard: `stall_timeout: 10.0s` detects frozen network streams cleanly and triggers bounded reconnects (`max_retries: 5`).
- Single-Model ONNX Thread Lock: `CameraManager` guards shared ONNX model inference via `threading.RLock()` across concurrent RTSP camera workers.

PART 15 - ENGINEERING CHALLENGES, DEBUGGING CHRONICLES, AND RESOLUTIONS

1. Challenge: 1.25 FPS Unusable CPU Throughput

- Root Cause: PyTorch `.pt` YOLO calls (101.6 ms) + RapidOCR `DB736` upscaling and enabled 180-deg angle classifier (556.5 ms) + redundant OCR on parked/ already-stored tracks.
- Resolution: Exported `.onnx` detectors (`_onnx_yolo.py`), adopted lightweight YOLOv8n `best_finetuned.onnx`, capped `det_limit_side_len=320`, `use_cls=false`, disabled CPU SR (`sr_threshold_px=0`), and added `stored_tracks` + `max_heavy_tracks_per_frame=2` -> Achieved **16.27 - 23.76 FPS**.

2. Challenge: `frame_skip=2` Silently Dropping Vehicles

- Root Cause: Skipping frames doubled inter-frame displacement, causing ByteTrack IoU mismatch and restarting `OCRFusion` vote buffers mid-pass.
- Resolution: Kept `frame_skip=1` + increased `lost_track_timeout=60` and lowered `minimum_matching_threshold=0.5`, recovering **6/6 vehicles**.

3. Challenge: Next.js 15 SSR React Hydration Errors & Tiny UI Legibility

- Root Cause: Browser password/form extensions injecting `fdprocessedid` + SSR/client mismatch and small CSS font tokens reported during dashboard testing.
- Resolution: Applied clean client hydration safeguards and scaled UI font hierarchy / paddings in `frontend/`, committed cleanly without breaking API flow.

4. Challenge: PARSeq Official ONNX Incompatible on CPU ORT

- Root Cause: Exported `parseq_official_tiny.onnx` uses `Where(16)` semantics raising `NotImplemented` on x86 `onnxruntime`.
- Resolution: Routed default CPU backend cleanly to `rapidocr` while retaining `parseq_official_tiny.engine` strictly for CUDA/TensorRT 10.16 targets.

5. Challenge: Audit Discovery of Training Data Leakage & Phantom 25,674 Claim

- Forensic Audit Discovery: Caught `training/dataset_converter.py` aliasing `val == train` in legacy `train/` run (`mAP50=0.9949` invalid) and proved `4,116` val in documented `25,674` count was a phantom 20% arithmetic duplicate.

PART 16 - PROJECT LIMITATIONS, EVIDENTIARY GAPS, AND AUDIT DISCLOSURE

Strictly Separating Verified vs Partially Tested vs Planned/Unverified Features:

[VERIFIED IN CODEBASE & EXECUTED]

- End-to-end single-stream CPU ONNX + RapidOCR + ByteTrack + Fusion + DB + API

+ Next.js 15 pipeline (`19.18 FPS` full 1500f clip, `16.27 FPS` live CAM001).

- Pre-store blacklist alert lead time (+188 ms to +1,127 ms before DB write).
- SQLite WAL migration, atomic `SAVEPOINT` VehicleProfile updates, 500-visit trajectory JSON cap, 41 REST/streaming routes, 629+ verified non-duplicate unit/integration/ASGI test passes across runs.

[PARTIALLY TESTED / STALE TEST DRIFT]

- Streamlit `dashboard_v2.py`: 55/58 AppTest pass; 3 failures are Streamlit 1.67.0 `imgs->image` key drift and hardcoded `Sept 12` fixture age.
- `test_parseq_dataset.py` (1 fail) & `test_super_resolution.py` (2 fails).
- `test_dataset_converter.py`: Timed out on full multi-GB disk copy; 1 passed.

[UNVERIFIED / ABSENT / NOT EXECUTED]

- **Simultaneous 5-Minute Dual-RTSP Campus Camera Live Validation**: Neither physical campus RTSP camera stream sustained 5-min stress test nor real-world per-camera FPS/RAM/CPU telemetry was executed in this workspace.
- **Labeled OCR Accuracy / CER Benchmark**: Zero quantitative ground-truth OCR accuracy score exists for either RapidOCR or PARSeq on Indian plates.
- **833-Image Fine-Tuning Dataset & Executed Notebook**: Zip file absent from repository; `Campus_ANPR_YOLO_Training.ipynb` saved unexecuted (`outputs: []`).
- **Unused** `postgres_schema.py` **DDL**: 6 normalized tables + 1 MV exist only as unreferenced schema DDL; runtime uses `models.py` 2-table schema.
- **Jetson TensorRT Execution**: `.engine` artifacts present on disk, untested on physical Jetson GPU hardware in this Windows validation environment.

PART 17 - DEPLOYMENT RECOMMENDATIONS AND FUTURE ENGINEERING ROADMAP

1. High-Priority Immediate Fixes (Before Final Academic Demo / Campus Go-Live):

- Execute a reproducible OCR benchmark script over a stratified sample of `OCR_data/` (`n >= 500` across MH/DL/UP/BR/TN/KA) measuring exact Character Error Rate (CER) and Full-Plate Accuracy for RapidOCR CPU.
- Fix `evaluate.py`'s JSON extraction bug on the 978-image test split so `mAP50`, `Precision`, `Recall`, and `mAP50-95` print cleanly alongside `fitness`.
- Fix `training/dataset_converter.py` L58-62 to split disjoint train/val folders (eliminating any risk of future `val == train` leakage).
- Commit the `~833`-image Campus dataset manifest/split summary + executed `.ipynb` outputs (or re-run validation locally against held-out CCTV frames).

2. Live Campus Hardware Validation Protocol:

- Execute a logged 5-minute simultaneous dual-RTSP run (`CAM001` + `CAM002`) capturing per-camera FPS, process CPU%, RSS memory, reconnection behavior on network unplug, IN/OUT confusion matrix against physical vehicle passes, and measured alert latency on a real watchlisted test vehicle.
- Deploy exported `.engine` artifacts (`yolov8n.engine`, `best.engine`, `parseq_official_tiny.engine`) on NVIDIA Jetson Orin Nano to benchmark GPU TensorRT PARSeq accuracy/FPS vs CPU RapidOCR.

3. Architectural Maturity Upgrades:

- Add API token / role-based authentication (`RBAC`) protecting blacklist/ whitelist mutations, DB clear, and camera control endpoints.

- Fix the 3 unit test drifts (`test_parseq_dataset.py`, `test_super_resolution.py`) and 3 Streamlit v1.67.0 `AppTest` assertions (`image` key + relative dates).
- Either archive or integrate `src/database/postgres_schema.py` cleanly so schema documentation reflects a single canonical database model.